# Envy-Freeness with Limited Subsidies under Negative Dichotomous Valuations

*Working draft — not for circulation*

Mainak

**What this document is.** The complete state of the work, including results that are proved and machine-checked but not yet worth putting in the report. Sections 1 to 3 are shared verbatim with `main.tex`; everything after them is parked material. When a parked section earns its place, move its file from `working/` back to `sections/` and uncomment the matching line in `main.tex`. Nothing here is stale: every claim was re-verified against the scripts on the date of its section.

## 1 Introduction

Discrete fair division studies the allocation of resources that cannot be fractionally assigned among agents with individual preferences, and sits at the interface of mathematical economics and computer science [BCE+16, End17]. The classical fairness criterion is *envy-freeness* [Fol67, Var74]: no agent should prefer the bundle assigned to another agent over her own. For indivisible items an envy-free allocation need not exist, as the standard one-item two-agent instance shows, and a large body of work therefore studies relaxations such as envy-freeness up to one good (EF1) [Bud11, LMMS04].

A second and complementary line of work retains *exact* envy-freeness and buys off the residual envy with money. Each agent $i$ receives, in addition to her bundle, a subsidy $p_i \geq 0$ supplied by a third party, and one asks for an allocation together with a subsidy vector under which every agent weakly prefers her own bundle-plus-subsidy to that of every other agent. The question is how much money this requires. Maskin [Mas87] answered it for unit-demand agents with $m = n$: under the scaling in which no agent values any single item above 1, a total subsidy of $n - 1$ suffices. Halpern and Shah [HS19] moved to the multi-demand setting with arbitrary $m$, gave the characterisation of *envy-freeable* allocations that the whole line now runs on, showed that a subsidy of $(n - 1)m$ is necessary and sufficient in the worst case when the allocation is handed to us, and conjectured that $n - 1$ suffices when we may choose the allocation. Brustle et al. [BDN+20] proved that conjecture in the stronger per-agent form — one dollar to each agent suffices for additive valuations, with the allocation simultaneously EF1 and balanced — and showed that for general monotone valuations $2(n - 1)$ per agent suffices, so that the total subsidy is $O(n^2)$ and, remarkably, *independent of the number of items*. Barman et al. [BKNS22] then improved the monotone bound by a factor of $n$ on the dichotomous subclass: when every marginal

$v_i(S \cup \{g\}) - v_i(S)$ lies in $\{0, 1\}$, there is an allocation whose accompanying subsidy is exactly 0 or 1 for each agent, hence at most $n - 1$ in total, computable in polynomial time in the value-oracle model. Their bound assumes neither additivity nor submodularity nor even subadditivity, and it is tight: with $n$ agents and a single unit-valued good, $n - 1$ agents must each be paid 1.

**Chores.** Every result above is about *goods*. This paper asks the mirrored question, in which every item is a *chore*: an object that an agent would rather not hold, so that acquiring it weakly decreases her utility. Chores are not an exotic variant. Shift rotations, on-call duty, teaching and refereeing load, committee service, maintenance tasks and the liabilities attached to an estate are all allocation problems in which the items are burdens rather than prizes, and in all of them money is already the standard instrument of repair — as overtime pay, hardship allowance, or a course-release budget. If anything the subsidy model is more natural for chores than for goods, since compensating somebody for taking on unwanted work needs less justification than paying somebody for the privilege of not receiving a prize.

Concretely, we study *negative dichotomous* valuations: for every agent $i$, every $S \subseteq M$ and every $g \in M \setminus S$,

$$v_i(S) - v_i(S \cup \{g\}) \ \in \ \{0, 1\}\,.$$

Equivalently, each marginal $v_i(S \cup \{g\}) - v_i(S)$ is 0 or $-1$: every item is a chore for every agent, and taking on one more chore costs an agent either nothing or exactly one unit. This is the exact mirror of the model of [BKNS22], and we impose no further structure — in particular the valuations need not be additive, submodular, or subadditive.

Binary marginals are the natural model whenever a task either lands on an agent as a fresh unit of burden or is absorbed by a commitment she has already made. An engineer already staffing an on-call window absorbs a second incident in that window at no extra cost; a referee already reading for a venue absorbs one more paper from the same batch; a driver already routed through a district absorbs an extra stop along the way. Whether a given chore is free depends in an arbitrary way on the rest of the agent's assignment, which is exactly why the model must tolerate non-additive and even non-subadditive valuations, and it is the counterpart of the scheduling example used by [BKNS22] to motivate dichotomous valuations for goods.

**Why this is not a change of sign.** It is tempting to expect that a chores result follows from the corresponding goods result by negating the valuations. It does not, for three reasons that we record here because they shape everything that follows.

First, money is one-signed. The subsidy vector is constrained to $p \in \mathbb{R}^n_+$, and negating an instance negates the valuations but leaves the subsidies where they are. The polyhedron of feasible subsidy vectors is therefore a genuinely different object in the two models, and no formal duality carries a bound across.

Second, the obvious transformation is available only for two agents. Writing $c_i := -v_i$ for the cost form of a negative dichotomous valuation, the complement $\tilde{v}_i(S) := c_i(M) - c_i(M \setminus S)$ is again dichotomous in the sense of [BKNS22], so the class is closed under complementation. But the complement transformation respects the allocation only when $M \setminus A_j = A_i$, that is,

only when $n = 2$; see Remark 9. For $n \geq 3$ there is no such reduction, and that is where the content of the problem lies.

Third, and most tellingly, envy flows the other way. Under goods, envy concentrates *into* the agent holding the valuable items: many agents envy one, and the tight instance of [BKNS22] pays $n - 1$ agents one dollar each so that they stop envying the single agent who received the good. Under chores, envy emanates *out of* the agent carrying the load: one agent envies many. Since the minimum subsidy to an agent is the weight of the heaviest directed path leaving her in the envy graph [HS19, Theorem 2], the two situations are not interchangeable, and one should not assume in advance that the same bound is the right target.

**What is already known, and what is open.** Two facts delimit the problem before any work is done, and we state them explicitly so that the contribution can be measured against them.

On the one hand, existence of a bounded subsidy is not in question. Negative dichotomous valuations are doubly monotone — every item is a chore for every agent, which is a special case of every item being a good or a chore for each agent — and they satisfy the normalisation $|v_i(S \cup \{e\}) - v_i(S)| \leq 1$ that [KMS+24, §2] imposes. The reduction of Kawase et al., which converts any EF1 allocation into an envy-free one, therefore applies off the shelf and yields a subsidy of at most $n - 1$ per agent and $n(n-1)/2$ in total [KMS+24, Theorem 1]. Two details make the containment work and are easy to get wrong. Their normalisation is two-sided, so it bounds chore disutility as well as value; and the EF1 they require is the two-sided form of Theorem 5, not the goods-only form of [HS19] — the two differ precisely when chores are present. An EF1 allocation is guaranteed to exist and to be computable in polynomial time for doubly monotone valuations, so the reduction has an input; for that step [KMS+24, §2] cites [LMMS04] and [BSV21], and the citation matters, because the envy-cycle argument of [LMMS04] does *not* port to chores unaltered and it is [BSV21] that supplies the repair.

On the other hand, the lower bound of the goods model survives intact.

**Example 1** (The $n - 1$ lower bound transfers)**.** Take $n$ agents and $m = n - 1$ chores, with $c_i(S) = |S|$ for every agent $i$ — identical, binary, additive costs, which are in particular negative dichotomous. Every complete allocation leaves at least one agent empty-handed. Under the balanced allocation, in which $n - 1$ agents hold one chore each and one agent holds nothing, each of the $n - 1$ loaded agents envies the empty agent by exactly 1 and no other envy is present, so the minimum subsidy assigns 1 to each loaded agent and 0 to the empty one, for a total of $n - 1$. No allocation does better: any complete allocation gives some agent a bundle of size $k \geq 1$ while some other agent is empty, and that agent's heaviest outgoing path already has weight $k$.

So the target statement is precisely the analogue of [BKNS22, Theorem 4].

**Conjecture 2.** *For every instance with negative dichotomous valuations there is an envy-free solution $(\mathcal{A}, p)$ with $p \in \{0, 1\}^n$, hence with total subsidy at most $n - 1$; and it can be computed in polynomial time given value-oracle access.*

Example 1 shows Conjecture 2 would be tight. Between the $n - 1$ per agent that [KMS+24] already gives and the $\{0, 1\}$ per agent being asked for lies a factor of $n$ in the total, exactly

the gap that [BKNS22] closed on the goods side. Our objective is to close it on the chores side or to exhibit an instance showing that it cannot be closed.

**Scope of this report.** Conjecture 2 is open. This report sets up the problem and stops there: Section 2 fixes the model and notation and records what transfers from the goods theory unchanged, and Section 3 isolates three structural facts about the chore problem — an arc-update lemma (Lemma 13) reducing the entire goods/chore asymmetry to two lines, a two-tier characterisation of $\{0, 1\}$ solutions (Proposition 14), and a size-shift transform (Theorem 15) exhibiting the chore problem as exactly the goods problem of [BKNS22] together with a cardinality-balancing requirement. The last of these is the precise sense in which Conjecture 2 does not follow from [BKNS22] as a black box.

Work towards the conjecture is in progress and is deliberately not reported here. Several special cases are settled and two natural proof strategies have been ruled out by explicit machine-checked instances; none of it is stated below until it is worth stating. The remainder of this working draft, from Section 4 onwards, carries that material.

**Related Work.** The subsidy line for goods is described above. Within it, the dichotomous subclass has attracted separate attention: Halpern and Shah [HS19] settle binary additive valuations with a total subsidy of $n - 1$, Goko et al. [GIK+21] obtain the analogous bound for binary submodular valuations together with a strategy-proof mechanism, and Barman et al. [BKNS22] subsume both by dropping every structural assumption beyond binary marginals. Dichotomous preferences are themselves a well-studied restriction in social choice [BMS05, BEF21]. On the computational side, Caragiannis and Ioannidis [CI21] show that minimising the subsidy is NP-hard and give a fully polynomial-time approximation scheme for a constant number of agents, and Narayan et al. [NSV21] study the welfare cost of achieving envy-freeness with transfers.

The paper closest to the present setting is that of Kawase et al. [KMS+24], the only work in the envy-freeness-with-subsidy line that admits chores at all. Working with doubly monotone valuations under the two-sided normalisation, it decouples the two jobs that [BDN+20] performed together: rather than constructing a specific allocation and showing that it is cheap to subsidise, it takes *any* EF1 allocation as input and bounds the subsidy needed to convert it, obtaining $n - 1$ per agent and $n(n - 1)/2$ in total. Since EF1 allocations are known to exist for doubly monotone valuations, the wider class comes along at no cost. That reduction is what makes the present question well-posed rather than open-ended, and it is the baseline we are trying to beat.

The complementary repair — no money, weaker fairness notion — is pursued for binary valuations by Bu et al. [BSY23], who show that EFX allocations always exist and can be computed in polynomial time for general binary valuations, thus extending the matroid-rank result of Babaioff et al. [BEF21]. Under binary valuations one therefore has a clean dichotomy on the goods side: either pay 0 or 1 per agent and obtain exact envy-freeness [BKNS22], or pay nothing and obtain EFX [BSY23]. Whether either branch survives the passage to chores is open; this paper addresses the first.

**The chores side.** Work on indivisible chores has largely developed alongside rather than inside the subsidy line, and the two have met less often than one might expect. Where money has been brought to chores, it has been in the service of *proportionality* rather than envy-freeness: the question studied is the total subsidy needed to bring every agent down to her proportional share of the burden, for which the best bounds under additive disutilities normalised to at most 1 per chore are $n/4$ and then $n/3 - 1/6$, recently obtained together with Pareto optimality by Garg et al. [GSW25]. That is a different fairness notion and a different accounting; it does not bound the subsidy needed for envy-freeness.

The no-money branch, by contrast, has a direct chore analogue of [BSY23]. Tao et al. [TWYZ23] study chores with binary marginals and show that for binary *additive* costs an allocation that is simultaneously EFX and Pareto optimal always exists and is computable in polynomial time. So on the chores side the second half of the binary dichotomy — pay nothing, obtain EFX — is established at least for additive costs, while the first half, pay 0 or 1 per agent and obtain exact envy-freeness, is what Conjecture 2 asks for. That money is genuinely needed here is not obvious a priori but is known: Bhaskar et al. [BSV21] show that deciding whether an exactly envy-free allocation of chores exists is NP-complete already for binary additive costs, and give a polynomial-time EF1 algorithm for chores and for doubly monotone instances.

Where the two lines have met is on the *additive* side, and recently. Lu et al. [LMS26] prove the goods-and-chores analogue of [BDN+20]: for additive utilities in which each item is a good for some agents and a chore for others, with every $|v_i(g)| \leq 1$, a subsidy of one dollar per agent suffices, the bound is tight, and the allocation is computable in polynomial time — hence $n - 1$ in total. This settles the additive chore problem completely.

It does not settle ours, and the reason is exactly the reason [BKNS22] was not a corollary of [BDN+20]. Dichotomous valuations are not additive and additive valuations are not dichotomous; the two classes are incomparable. The four corners are

| | goods | chores |
|---|---|---|
| additive | [BDN+20] | [LMS26] |
| dichotomous | [BKNS22] | **Conjecture 2** |

each of the three settled corners giving one dollar per agent and $n - 1$ in total. The present question is the fourth. Note also that [LMS26] does not displace [KMS+24] as the baseline of Section 1: for valuations that are dichotomous but not additive, $n - 1$ per agent from [KMS+24] remains the best published bound.

We are not aware of any treatment of envy-freeness with subsidy for chores under dichotomous costs. This is the outcome of a literature search rather than a proof of novelty — [LMS26] appeared in July 2026, which is a reminder of how quickly the surrounding corners are being filled in — and it should be repeated before the paper is circulated.

## 2 Notation and Preliminaries

We study the allocation of $m$ indivisible items among $n$ agents, with subsidies. Write $N = [n]$ for the set of agents and $M$ for the set of items, $|M| = m$. Each agent $i \in N$ has a valuation

$v_i : 2^M \to \mathbb{R}$ with $v_i(\emptyset) = 0$, and we represent an instance by the tuple $\langle N, M, \{v_i\}_{i \in N} \rangle$. Algorithms operate in the standard *value-oracle* model: for each $v_i$ we assume only an oracle that returns $v_i(S)$ for a queried set $S \subseteq M$, and running time is measured in $n$, $m$ and the number of oracle calls.

An *allocation* $\mathcal{A} = (A_1, \ldots, A_n)$ is an ordered tuple of pairwise disjoint *bundles*, with $A_i$ assigned to agent $i$. The allocation is *complete* if $\bigcup_{i \in N} A_i = M$ and *partial* otherwise. The ordering matters throughout: much of the theory below concerns permuting a fixed family of bundles among the agents, and for a permutation $\sigma \in \mathbb{S}_n$ we write $\mathcal{A}_\sigma := (A_{\sigma(1)}, \ldots, A_{\sigma(n)})$ for the allocation in which agent $i$ receives $A_{\sigma(i)}$. The utilitarian welfare of $\mathcal{A}$ is $\mathrm{SW}(\mathcal{A}) = \sum_{i \in N} v_i(A_i)$.

**Definition 3** (Envy-Freeness)**.** *An allocation $\mathcal{A} = (A_1, \ldots, A_n)$ is* envy-free *(EF) iff $v_i(A_i) \geq v_i(A_j)$ for all agents $i, j \in N$.*

Envy-free allocations of indivisible items need not exist, so we allow a third party to supplement the allocation with money. Agents have quasi-linear utilities: agent $i$ receiving bundle $A_i$ and subsidy $p_i$ enjoys utility $v_i(A_i) + p_i$, with money entering linearly and at the same exchange rate for every agent.

**Definition 4** (Envy-Free Solution)**.** *An allocation $\mathcal{A} = (A_1, \ldots, A_n)$ and a subsidy vector $p = (p_1, \ldots, p_n) \in \mathbb{R}^n_+$ constitute an* envy-free solution *$(\mathcal{A}, p)$ iff $v_i(A_i) + p_i \geq v_i(A_j) + p_j$ for all $i, j \in N$.*

An allocation $\mathcal{A}$ is *envy-freeable* if some $p \in \mathbb{R}^n_+$ makes $(\mathcal{A}, p)$ an envy-free solution; this is a property of the allocation alone. Following [BKNS22] we write $\mathrm{M}(p) := \left\{ i \in N \mid p_i \geq p_j \text{ for all } j \in N \right\}$ for the set of maximally subsidised agents.

We will occasionally need the standard relaxation of envy-freeness. Note that two inequivalent forms of it appear in this literature: the goods-only form of [HS19], which removes one item from the envied bundle, and the two-sided form used by [KMS+24], which removes one item from *either* bundle. They coincide when all items are goods and differ once chores are present, and only the two-sided form supports the doubly monotone results. We adopt the two-sided form.

**Definition 5** (EF1, two-sided form)**.** *An allocation $\mathcal{A}$ is* envy-free up to one item *(EF1) iff for all $i, j \in N$ there exists $X \subseteq A_i \cup A_j$ with $|X| \leq 1$ such that $v_i(A_i \setminus X) \geq v_i(A_j \setminus X)$.*

## 2.1 Negative Dichotomous Valuations

**Definition 6** (Negative Dichotomous Valuation)**.** *A valuation $v_i : 2^M \to \mathbb{R}$ with $v_i(\emptyset) = 0$ is* negative dichotomous *iff*

$$v_i(S) - v_i(S \cup \{g\}) \in \{0, 1\} \qquad \text{for every } S \subseteq M \text{ and every } g \in M \setminus S,$$

*equivalently iff every marginal $\Delta_i^+(S, g) := v_i(S \cup \{g\}) - v_i(S)$ lies in $\{-1, 0\}$.*

Every item is thus a chore for every agent: $v_i$ is non-increasing, so $v_i(S) \geq v_i(T)$ whenever $S \subseteq T$. We stress that nothing further is assumed — the valuations need not be additive,

submodular, or subadditive — which is the generality at which [BKNS22] works on the goods side.

It is often more convenient to work with the non-negative *cost form*

$$c_i \;:=\; -v_i, \qquad \text{so} \qquad c_i(\emptyset) = 0, \quad c_i(S) \le c_i(T) \text{ for } S \subseteq T, \quad c_i(S \cup \{g\}) - c_i(S) \in \{0,1\}\,.$$

**Definition 7** (Dichotomous cost function)**.** *A set function $c : 2^M \to \mathbb{R}$ is* dichotomous *if $c(\emptyset) = 0$ and every marginal $c(S \cup \{g\}) - c(S)$ lies in $\{0,1\}$.*

Monotonicity is not a separate hypothesis: marginals in $\{0,1\}$ are non-negative, so a dichotomous $c$ is automatically non-decreasing. We nonetheless list it above because it is the property most often used directly.

**Remark 8** (Three adjectives that are routinely confused)**.** *Monotone* means only $c(S) \le c(T)$ for $S \subseteq T$, with no bound whatever on how large an increment can be; it is far weaker than Theorem 7 and, on its own, supports almost none of the arguments below. *Dichotomous* adds the binary-marginal condition and is the class of this paper. *Binary additive* is the strictly smaller class $c_i(S) = |S \cap D_i|$ for a set $D_i$ of disliked chores, i.e. additive with per-item costs in $\{0,1\}$; it is the case settled by [LMS26].

A warning about the literature: several papers say "binary valuations" for what is called dichotomous here — [BSY23] and [TWYZ23] both do — and reserve "binary additive" for the additive subclass. When a bound is quoted, check which is meant, since the classes are not the same and the gap between them is where this paper lives.

The correspondence $v_i \leftrightarrow c_i$ is a bijection between negative dichotomous valuations and dichotomous cost functions in the sense of [BKNS22]: the class we study is exactly the pointwise negation of the class they study. In cost form an envy-free solution is the requirement $c_i(A_i) - p_i \le c_i(A_j) - p_j$ for all $i, j$, read as "my own burden net of my compensation is no worse than what I perceive of yours". We use whichever of $v_i$ and $c_i$ is clearer locally and always state which.

The negation, however, does not carry results across, for the reasons given in Section 1. The following remark isolates the one case in which a transformation does exist, and shows why it is uninformative.

**Remark 9** (Complementation, and why it stops at $n = 2$)**.** For a negative dichotomous $v_i$ with cost form $c_i$, define the *complement* $\tilde{v}_i(S) := c_i(M) - c_i(M \setminus S)$. Then $\tilde{v}_i(\emptyset) = 0$ and, for $g \notin S$, writing $T := M \setminus (S \cup \{g\})$,

$$\tilde{v}_i(S \cup \{g\}) - \tilde{v}_i(S) \;=\; c_i(M \setminus S) - c_i(T) \;=\; c_i(T \cup \{g\}) - c_i(T) \;\in\; \{0,1\}\,,$$

so $\tilde{v}_i$ is dichotomous in the sense of [BKNS22]. The class is therefore closed under complementation, without any submodularity assumption. Now let $n = 2$ and let $\mathcal{A} = (A_1, A_2)$ be complete, so that $M \setminus A_2 = A_1$ and $M \setminus A_1 = A_2$. Then $-c_i(A_i) = \tilde{v}_i(A_j) - c_i(M)$ and $-c_i(A_j) = \tilde{v}_i(A_i) - c_i(M)$, so $(\mathcal{A}, p)$ is an envy-free solution for the chores instance $\{v_i\}$ if and only if $((A_2, A_1), p)$ is an envy-free solution for the goods instance $\{\tilde{v}_i\}$: swap the bundles and leave each subsidy attached to its agent. Two-agent chore division is thus exactly two-agent goods division, and [BKNS22] settles it. The step $M \setminus A_j = A_i$ has no analogue for $n \ge 3$, where the complement of one bundle is a union of several others, and no reduction of this kind is available.

## 2.2 The Envy Graph and the Halpern–Shah Characterisation

For an allocation $\mathcal{A}$, the *envy graph* $G_{\mathcal{A}}$ is the complete weighted directed graph on vertex set $N$ in which the arc $(i, j)$ carries weight

$$w_{\mathcal{A}}(i, j) \;:=\; v_i(A_j) - v_i(A_i) \;=\; c_i(A_i) - c_i(A_j),$$

the envy that agent $i$ has towards agent $j$. The weight of a directed path $P$ is $w_{\mathcal{A}}(P) = \sum_{(i,j)\in P} w_{\mathcal{A}}(i, j)$, and $\ell_{\mathcal{A}}(i)$ denotes the maximum weight of any path in $G_{\mathcal{A}}$ starting at $i$, the empty path of weight 0 included.

The two results below are the engine of the entire subsidy line. Both are stated by Halpern and Shah [HS19] for additive valuations, but their proofs use only the arc weights and permutations of the bundles — neither additivity nor monotonicity nor any sign condition on $v_i$ enters — and [BKNS22] invokes them in exactly this generality for (non-additive) dichotomous valuations. They therefore apply verbatim to negative dichotomous valuations, and we use them without further comment.

**Theorem 10** ([HS19])**.** *For any allocation $\mathcal{A} = (A_1, \ldots, A_n)$, the following are equivalent.*

*(i)* *$\mathcal{A}$ is envy-freeable.*

*(ii)* *$\mathcal{A}$ maximises utilitarian welfare across all reassignments of its own bundles among the agents: for every permutation $\sigma$ over $N$, $\sum_{i\in N} v_i(A_i) \geq \sum_{i\in N} v_i(A_{\sigma(i)})$.*

*(iii)* *The envy graph $G_{\mathcal{A}}$ has no positive-weight directed cycle.*

**Theorem 11** ([HS19])**.** *For any envy-freeable allocation $\mathcal{A}$, the subsidy vector $p^*$ given by $p^*_i := \ell_{\mathcal{A}}(i)$ realises an envy-free solution $(\mathcal{A}, p^*)$, and $p^*_i \leq p_i$ for every $i \in N$ and every $p$ such that $(\mathcal{A}, p)$ is an envy-free solution. It can be computed in strongly polynomial time by running an all-pairs longest-path computation on $G_{\mathcal{A}}$.*

Condition (ii) of Theorem 10 has the standing consequence that an envy-freeable allocation can always be manufactured from an arbitrary one: fix the bundles and reassign them according to a maximum-weight perfect matching between agents and bundles. What is at issue is never the existence of an envy-freeable allocation, only the size of the subsidy that Theorem 11 then charges for it.

Specialising to our model gives the following, which we use throughout.

**Observation 12** (Integrality)**.** *Let $\{v_i\}_{i\in N}$ be negative dichotomous. Then $v_i(S) \in \mathbb{Z}_{\leq 0}$ for every $i$ and every $S \subseteq M$; every arc weight $w_{\mathcal{A}}(i, j)$ is an integer; and for every envy-freeable allocation $\mathcal{A}$ the minimum subsidy vector satisfies $p^* \in \mathbb{Z}^n_{\geq 0}$.*

**Proof** Fix $i$ and $S = \{g_1, \ldots, g_k\}$. Telescoping from $v_i(\emptyset) = 0$ along any ordering of $S$ writes $v_i(S)$ as a sum of $k$ marginals, each of which lies in $\{-1, 0\}$ by Definition 6; hence $v_i(S) \in \mathbb{Z}$ and $-k \leq v_i(S) \leq 0$. Consequently each arc weight $w_{\mathcal{A}}(i, j) = v_i(A_j) - v_i(A_i)$ is a difference of integers, and the weight of any directed path, being a finite sum of arc weights, is an integer. By Theorem 11, $p^*_i = \ell_{\mathcal{A}}(i)$ is the maximum such weight over paths starting at $i$, and it is non-negative because the empty path has weight 0. □

Observation 12 is what makes the target statement — a subsidy vector in $\{0,1\}^n$ — a statement about a *bound* rather than about rounding: once $p^*$ is known to be integral, showing $p_i^* \le 1$ for every $i$ is the same as showing $p^* \in \{0,1\}^n$. The same observation underlies the corresponding step in [BKNS22], where the arc weights of a dichotomous instance are likewise integers.

# 3 Structure of the Chore Problem

Throughout this section we work in cost form, so $w_{\mathcal{A}}(i,j) = c_i(A_i) - c_i(A_j)$.

## 3.1 The arc-update table, and the whole of the asymmetry

Every incremental algorithm in this literature adds one item to one bundle and then asks what happened to the envy graph. The answer is two lines long, and comparing those two lines against their goods counterparts accounts for the entire difference between the present problem and that of [BKNS22].

For an allocation $\mathcal{A}$, an agent $x$ and an unallocated chore $g \notin \bigcup_i A_i$, write

$$Z^x \;:=\; (A_1, \dots, A_x \cup \{g\}, \dots, A_n)$$

for the allocation obtained by giving $g$ to $x$, and put

$$\delta_x \;:=\; c_x(A_x \cup \{g\}) - c_x(A_x), \qquad \delta_{i,x} \;:=\; c_i(A_x \cup \{g\}) - c_i(A_x),$$

both of which lie in $\{0,1\}$. So $\delta_x$ is what the chore costs $x$ herself, and $\delta_{i,x}$ is what it costs in $i$'s opinion.

**Lemma 13** (Arc update)**.** *For all $i, j \in N$,*

$$w_{Z^x}(i,j) = w_{\mathcal{A}}(i,j) \quad (i,j \neq x), \qquad w_{Z^x}(x,j) = w_{\mathcal{A}}(x,j) + \delta_x, \qquad w_{Z^x}(i,x) = w_{\mathcal{A}}(i,x) - \delta_{i,x}.$$

**Proof** Only $A_x$ changes, so an arc between two agents other than $x$ has both endpoints untouched. For an arc leaving $x$, $w_{Z^x}(x,j) = c_x(A_x \cup \{g\}) - c_x(A_j) = w_{\mathcal{A}}(x,j) + \delta_x$. For an arc entering $x$, $w_{Z^x}(i,x) = c_i(A_i) - c_i(A_x \cup \{g\}) = w_{\mathcal{A}}(i,x) - \delta_{i,x}$. □

The mirror statement for goods is obtained by replacing costs with values: there the arcs *out of* the receiving agent fall and the arcs *into* her rise. The consequences are opposite in a way that matters.

| | goods [BKNS22] | chores (here) |
|---|---|---|
| arcs out of the receiver | fall | **rise by** $\delta_x$ |
| arcs into the receiver | **rise** | fall by $\delta_{i,x}$ |
| so the receiver must be | *maximally* subsidised | *minimally* subsidised |
| which is guaranteed by | $\mathrm{M}(p) \neq \emptyset$ | $p_x = 0$ for some $x$ |

Both guarantees hold trivially — some agent is always maximally subsidised, and some agent always has $p_x = 0$ under the minimum subsidy vector. The asymmetry is subtler than the table suggests, and it is what Theorem 30 exploits. Under goods, the harm done by an insertion is confined to paths *ending* at the receiver, and requiring $x \in \mathrm{M}(p)$ caps every such path at $p_u - p_x \le 0$ in one stroke. Under chores, the harm is done to paths *leaving* the receiver, and $p_x = 0$ caps only the paths that *start* at $x$. A path that merely passes through $x$ — entering along an arc from some $i$ with $\delta_{i,x} = 0$ and leaving along an arc that has just risen by $\delta_x = 1$ — is not controlled by any condition on $p_x$. That gap is not a defect of the analysis; Section 6 shows it is realised.

### 3.2 The shape of a $\{0,1\}$ solution

**Proposition 14** (Two-tier characterisation)**.** *Let $p \in \{0,1\}^n$ and put $S := \{i \in N \mid p_i = 1\}$. Then $(\mathcal{A}, p)$ is an envy-free solution if and only if*

*(a) $c_i(A_i) \le c_i(A_j)$ whenever $i, j$ lie both in $S$ or both in $N \setminus S$;*

*(b) $c_i(A_i) \le c_i(A_j) + 1$ for $i \in S$ and $j \in N \setminus S$;*

*(c) $c_i(A_i) + 1 \le c_i(A_j)$ for $i \in N \setminus S$ and $j \in S$.*

**Proof** In cost form Definition 4 reads $c_i(A_i) \le c_i(A_j) + p_i - p_j$ for all $i, j$. Substituting the three possible values $0, 1, -1$ of $p_i - p_j$ gives (a), (b) and (c) respectively. □

The reading is worth keeping in mind when designing an algorithm. The unpaid agents are exactly envy-free among themselves and *strictly* prefer their own bundle to that of every paid agent, by a full unit; the paid agents are exactly envy-free among themselves and envy-free up to one unit towards the unpaid. This is a certificate format, and it also suggests choosing the partition $S \mid N \setminus S$ *first* and allocating afterwards, rather than building the allocation up one chore at a time. That is the one place where the incremental template killed in Section 6 is not obviously an obstacle. In the proof of Theorem 18 the set $S$ turns out to be exactly the agents holding a $\lceil q/n \rceil$-share of the universally disliked chores.

### 3.3 The size-shift transform

Remark 9 showed that complementation relates the chore and goods models only for $n = 2$. There is a second transform, available for every $n$, which does not solve the problem but says precisely how far it is from being a corollary of [BKNS22].

**Theorem 15** (Size shift)**.** *For a negative dichotomous instance with cost functions $c_i$, define $\tilde{v}_i(S) := |S| - c_i(S)$. Then each $\tilde{v}_i$ is a dichotomous* goods *valuation in the sense of [BKNS22], and for every allocation $\mathcal{A}$ and all $i, j$,*

$$w_{\mathcal{A}}(i,j) \;=\; \tilde{w}_{\mathcal{A}}(i,j) \;+\; |A_i| - \big|A_j\big|,$$

*where $\tilde{w}_{\mathcal{A}}$ denotes the envy graph of the goods instance $\{\tilde{v}_i\}$. Consequently every* cycle *has the same weight in both graphs — so $\mathcal{A}$ is envy-freeable for the chores $\{c_i\}$ if and only if it is envy-freeable for the goods $\{\tilde{v}_i\}$ — while for a path $P$ from $u$ to $v$,*

$$w_{\mathcal{A}}(P) \;=\; \tilde{w}_{\mathcal{A}}(P) + |A_u| - |A_v|, \qquad \textit{hence} \qquad \ell_{\mathcal{A}}(u) \;=\; \max_{v \in N} \Big[\, \tilde{d}_{\mathcal{A}}(u,v) + |A_u| - |A_v| \,\Big],$$

*with $\tilde{d}_{\mathcal{A}}(u,v)$ the maximum weight of a path from $u$ to $v$ in the goods graph and $\tilde{d}_{\mathcal{A}}(u,u) = 0$.*

**Proof** $\tilde{v}_i(\emptyset) = 0$, and for $g \notin S$, $\tilde{v}_i(S \cup \{g\}) - \tilde{v}_i(S) = 1 - (c_i(S \cup \{g\}) - c_i(S)) \in \{0,1\}$, so $\tilde{v}_i$ is dichotomous. For the identity,

$$\tilde{w}_{\mathcal{A}}(i,j) = \tilde{v}_i(A_j) - \tilde{v}_i(A_i) = \left(\left|A_j\right| - c_i(A_j)\right) - \left(|A_i| - c_i(A_i)\right) = w_{\mathcal{A}}(i,j) - |A_i| + \left|A_j\right|.$$

Summing along a path $(i_1, \ldots, i_r)$, the terms $|A_{i_t}| - \left|A_{i_{t+1}}\right|$ telescope to $\left|A_{i_1}\right| - |A_{i_r}|$; along a cycle they telescope to 0. The characterisation of envy-freeability by the absence of positive cycles (Theorem 10) then transfers, and the formula for $\ell_{\mathcal{A}}(u)$ follows from Theorem 11. □

**Remark 16** (Why Conjecture 2 is not a corollary of [BKNS22])**.** Theorem 15 says the chore problem is the goods problem of [BKNS22] *plus a cardinality-balancing requirement*. A sufficient condition falls straight out: since $\tilde{d}_{\mathcal{A}}(u,v) \le \tilde{p}_u - \tilde{p}_v$ for any envy-eliminating $\tilde{p}$, an allocation solving the goods instance in which the quantity $|A_i| + \tilde{p}_i$ has spread at most 1 across agents satisfies $\ell_{\mathcal{A}} \le 1$. But [BKNS22] offers no control whatever on bundle cardinalities — its output need not even be EF1, by its own Appendix C — so the requirement is not met by invoking it as a black box. Nor can its algorithm simply be re-run with a balancing tie-break, because Theorem 30 rules out the entire item-by-item template on which it is built. The transform also shows the chore problem is at least as expressive as the goods problem, so no easier.

## 4 Our Results

The target is Conjecture 2, stated in Section 1: an envy-free solution with $p \in \{0,1\}^n$ and hence total subsidy at most $n - 1$. Example 1 supplies the matching lower bound, so it would be tight. By Observation 12 the conjecture is equivalent to the purely graph-theoretic statement that some allocation $\mathcal{A}$ has no positive-weight cycle in $G_{\mathcal{A}}$ and satisfies $\ell_{\mathcal{A}}(i) \le 1$ for every agent $i$.

We do not settle Conjecture 2. What we contribute is a map of the problem: two solved special cases, three structural tools, and — the substance of the paper — two theorems showing that the two most natural proof strategies both fail, one of them the strategy that succeeds on the goods side.

**Solved cases.** Conjecture 2 holds, constructively and in polynomial time, for **binary additive** costs (Theorem 18) and for **identical** costs (Theorem 20). The case $n = 2$ follows from [BKNS22] through the complementation of Remark 9. The binary additive proof is the one to imitate: dump every chore on somebody who finds it free, then balance whatever nobody wants. Its bound telescopes along paths in the envy graph, which is the mechanism all three tools below try to reproduce.

**Structural tools** (Section 3). The arc-update table of Lemma 13 isolates the entire difference between the goods and chore problems in two lines. Proposition 14 characterises $\{0,1\}$ solutions as a two-tier structure, which is the natural certificate format and suggests choosing the tiers before allocating. Theorem 15 exhibits the chore problem as *exactly* the goods

problem of [BKNS22] plus a cardinality-balancing requirement, which is the precise sense in which Conjecture 2 is not a corollary of [BKNS22].

**First negative result** (Section 6). Item-by-item insertion — allocate one chore at a time, never move an already-allocated chore, and repair only by permuting whole bundles — is the template of [BKNS22]. Theorem 30 exhibits a three-agent instance on which that template provably cannot be completed: a legal partial state with $p \in \{0,1\}^3$ and an unallocated chore such that *every* agent who receives it forces a subsidy of 2, while the full instance admits a zero-subsidy allocation. Any proof of Conjecture 2 must therefore be allowed to re-open bundles that have already been allocated. We state this bluntly because the template is seductive and patching the insertion invariant is a dead end, not an unsolved exercise.

**Second negative result** (Section 7). The natural repair is to stop building the allocation incrementally and instead select a good allocation globally, the obvious candidate being one that minimises total cost. This suggests the following strengthening.

**Conjecture 17.** *The allocation in Conjecture 2 may in addition be taken* utilitarian-optimal*, i.e. minimising* $\sum_{i \in N} c_i(A_i)$ *over all allocations.*

Proposition 33 refutes Conjecture 17: there is a three-agent, four-chore instance whose *unique* utilitarian-optimal allocation requires a subsidy of 2, although another allocation — of strictly higher total cost — requires no subsidy at all. Utilitarian optimality is therefore not merely an unhelpful restriction but an actively wrong one, and the programme of searching for a tie-breaking rule inside the utilitarian-optimal set is unsound as posed.

**A third approach, still open** (Section 8). Both failures above are failures of *timing*: insertion commits a chore to an owner and cannot take it back, and utilitarian optimality commits to the wrong allocation outright. Section 8 changes coordinates so that ownership is decided only by the last move that touches a chore. Theorem 38 re-expresses the chore instance as a *goods* instance on an item set carrying $n - 1$ copies of each chore, with identical envy graphs, so the dichotomous class is closed under the duality and [BKNS22] applies to the replica verbatim. Corollary 40 then reduces Conjecture 2 to a single residual constraint — *coverage*, that no agent receive two copies of one type. Read dynamically this is the *peel* process, in which the orientation of [BKNS22] is restored because the quantity handed out is relief rather than burden, and the witness of Theorem 30 is handled with the invariant intact. The approach is not known to work: Proposition 47 exhibits legal states from which no legal continuation exists, so what remains is to find a peel rule that never enters one.

**A third negative result** (Section 9). Given Corollary 40, the natural move is to change the numbers so that a coverage violation becomes unattractive, and thereby reuse [BKNS22] as a black box. Section 9 closes that at both levels. No dichotomous reweighting of the replica instance separates coverage from non-coverage (Theorems 58 and 59, the latter exhaustive over all $990^3$ candidate triples), and no inflation of item weights can steer the algorithm of [BKNS22], because its repair subroutine provably runs where every candidate's marginal is 0 and a duplicate moves no arc at all (Theorem 61, Corollary 62). Both failures have one

cause, named in Section 9.6: a rule fixed before the allocation is chosen cannot act on a quantity the allocation determines. Coverage must be enforced by the procedure that builds the allocation, not by the objective it is scored against.

**A working algorithm** (Section 11). The size-shift transform of Theorem 15 is an involution (Lemma 75), so the chore problem restates as a pure *goods* question, Target G, with no chores or scheduling in it (Proposition 77). Running [BKNS22]'s own algorithm on that goods instance fails for a structural reason — item-by-item insertion can lock two items into the same bundle permanently, verified by an exhaustive backtracking search that finds a 0-spread allocation *unreachable* by any legal execution of the algorithm (Section 11.3). Abandoning insertion for a construct-and-repair procedure — one pass of iterated maximum-weight perfect matching, repaired by cardinality-preserving single-item swaps, restarted on failure — has **zero failures across** 389,215 **test instances**, including every hard instance that defeated Approaches 1 through 5.

Four properties of that algorithm are now *theorems*: its outputs are certified (Theorem 81), it terminates in polynomial time $O(n^3m^3\tau + n^4m^3)$ (Theorem 82), its search space is genuinely the unordered balanced partitions (Lemma 80), and — the main theoretical result of this line — **it is complete for two agents** (Theorem 83): every two-agent dichotomous instance admits a balanced split of spread at most 1, found constructively in $O(m\tau)$ time with no restarts, by a discrete intermediate-value argument on the aggregate $u = \tilde{v}_1 + \tilde{v}_2$ along swap paths. This strengthens even the known $n = 2$ chore result, adding balancedness that the complementation route does not provide. For $n \geq 3$, completeness remains the one open statement (Section 11.8).

**Evidence** (Section 12). Conjecture 2 survives exhaustive verification for $n = m = 3$ over all 9,880 instances up to agent symmetry, and randomised search over some $3.5 \times 10^4$ further instances. Section 12 also explains why that evidence is weaker than the counts suggest, and what would strengthen it.

## 5 Solved Cases

Conjecture 2 holds in three restrictions of the model, by explicit polynomial-time constructions, and on one further class of instances cut out by a certificate rather than by a restriction on the valuations (Section 5.4). The first three proofs run the same way: bound $w_{\mathcal{A}}(i, j)$ by a difference $\phi_i - \phi_j$ of a per-agent potential, so that the bound telescopes to $\phi_{i_1} - \phi_{i_r}$ along a path and to 0 around a cycle. Envy-freeability and the $\{0, 1\}$ bound then both follow from controlling the spread of $\phi$. Reproducing that mechanism in general is the open problem.

**What is and is not new here.** This section should be read with its overlap against the literature stated plainly, because two of the three cases are largely or wholly not ours.

- **Binary additive costs** (Theorem 18) are *subsumed* by [LMS26], which settles all additive goods-and-chores instances. See Remark 19. The statement is not new; only the

potential-and-telescoping mechanism is retained, because that is what the general case has to reproduce.

- **Identical costs** (Theorem 20) are *not* subsumed, for the reason given in Remark 21 — but the result is elementary and is best read as a warm-up.
- **Two agents** is [BKNS22, Theorem 4]. Only the reduction carrying it to chores (Remark 9) is ours.
- **The uniformly balanced class** (Section 5.4) is new, and is the only case here not implied by a cited paper. It is a certificate rather than an algorithm, and it is incomplete.

Nothing in this section, in other words, is load-bearing. The substance of the paper is in Sections 6, 7, 9 and 10, which are negative, and in Sections 3 and 8, which are structural.

### 5.1 Binary additive costs

Let $c_i(S) = |S \cap D_i|$ for a set $D_i \subseteq M$ of chores that agent $i$ dislikes; every other chore is free for her. Call a chore *universally bad* if it lies in $D_i$ for every $i$, write $U$ for the set of universally bad chores and $q := |U|$.

**Theorem 18.** *For binary additive costs, the following allocation is envy-freeable with minimum subsidy $p^* \in \{0,1\}^n$, hence total subsidy at most $n-1$, and it is computable in time $O(nm)$.*

> *Give each chore $g \notin U$ to some agent $i$ with $g \notin D_i$; at least one exists, by definition of $U$. Distribute $U$ as evenly as possible, so that $u_i := |A_i \cap U| \in \{\lfloor q/n \rfloor, \lceil q/n \rceil\}$ for every $i$.*

*The bound is tight, by Example 1.*

**Proof** By construction every chore in $A_i \setminus U$ is free for $i$, so $c_i(A_i) = |A_i \cap D_i| = u_i$. For any $j$ we have $A_j \cap U \subseteq A_j \cap D_i$, since $U \subseteq D_i$, and therefore $c_i(A_j) = |A_j \cap D_i| \geq u_j$. Hence

$$w_{\mathcal{A}}(i,j) \;=\; c_i(A_i) - c_i(A_j) \;\leq\; u_i - u_j.$$

Summing along a path $P = (i_1, \ldots, i_r)$ the right-hand side telescopes:

$$w_{\mathcal{A}}(P) \;\leq\; u_{i_1} - u_{i_r} \;\leq\; \lceil q/n \rceil - \lfloor q/n \rfloor \;\leq\; 1.$$

Around a cycle the same computation gives $w_{\mathcal{A}}(C) \leq 0$, so $G_{\mathcal{A}}$ has no positive-weight cycle and $\mathcal{A}$ is envy-freeable by Theorem 10. By Theorem 11, $p^*_i = \ell_{\mathcal{A}}(i) \leq 1$, and Observation 12 forces $p^* \in \{0,1\}^n$. Since some agent receives $\lfloor q/n \rfloor$ universally bad chores and hence requires no subsidy, the total is at most $n-1$. Example 1 is binary additive with $D_i = M$ for every $i$, so the bound cannot be improved. □

**Remark 19** (Superseded, and by how much)**.** Theorem 18 is subsumed by [LMS26], which appeared in July 2026 and gives one dollar per agent for *all* additive goods-and-chores instances with $|v_i(g)| \leq 1$; binary additive costs are the special case $v_i(g) \in \{-1, 0\}$. We keep the proof because the mechanism, not the statement, is what the general case needs: the potential $\phi_i = u_i$ and its telescoping along paths is the only device in this paper that produces a $\{0,1\}$ bound from scratch, and the general conjecture is precisely the problem of finding the right $\phi$. Theorem 20 and the $n = 2$ case are *not* subsumed, since dichotomous costs need not be additive.

The construction is a two-phase version of the shape the general case must reproduce: *dump every chore on somebody who finds it free, then balance whatever nobody wants*. The obstruction to running the same argument for general negative dichotomous costs is that "free for $i$" becomes context dependent. A chore may be free on top of $A_j$ and cost a full unit on top of $A_i$, so there is no fixed set $D_i$ to compute with and no fixed potential $u_i$ to telescope against. Theorem 30 shows that this is a genuine obstacle rather than a bookkeeping nuisance.

### 5.2 Identical costs, and two agents

**Theorem 20.** *Suppose $c_1 = \cdots = c_n = c$. Then every allocation is envy-freeable, and greedily adding each chore to a currently cheapest bundle yields $p^* \in \{0,1\}^n$ in time $O(m(n+\tau))$, where $\tau$ is the cost of one oracle call.*

**Proof** For any permutation $\sigma$ we have $\sum_i c(A_{\sigma(i)}) = \sum_i c(A_i)$, since both sides sum the same function over the same multiset of bundles. So condition (ii) of Theorem 10 holds with equality for every allocation, and every allocation is envy-freeable. Moreover $w_{\mathcal{A}}(i,j) = c(A_i) - c(A_j)$ telescopes *exactly* along a path $P = (i_1, \ldots, i_r)$, giving $w_{\mathcal{A}}(P) = c(A_{i_1}) - c(A_{i_r})$ and hence

$$\ell_{\mathcal{A}}(u) \;=\; c(A_u) - \min_{j \in N} c(A_j).$$

It therefore suffices to keep $\max_i c(A_i) - \min_i c(A_i) \le 1$. This is an invariant of the greedy rule. It holds vacuously at the empty allocation. Suppose it holds before a step, so that $c(A_i) \in \{\mu, \mu+1\}$ for every $i$, where $\mu$ is the current minimum, and let the next chore go to a bundle attaining $\mu$. Its cost becomes $\mu$ or $\mu+1$, because marginals lie in $\{0,1\}$; every other bundle is unchanged. So all costs still lie in $\{\mu, \mu+1\}$ and the invariant survives. The bound $p^* \in \{0,1\}^n$ follows with Observation 12. □

**Remark 21** (Why this one survives the literature)**.** Theorem 20 is not covered by [LMS26], because identical dichotomous costs need not be additive. The instance $c_i(S) = \max(0, |S|-1)$ for every $i$ is identical and dichotomous, with marginals $0, 1, 1, \ldots$, and is supermodular rather than additive — it is the very cost function that drives the obstruction of Theorem 30. Nor does [BKNS22] cover it, that being a goods result, and [KMS+24] gives only $n-1$ per agent on this class. So the statement is new, in the narrow sense that no cited paper implies it.

It is also easy, and should not be oversold. With identical costs every agent agrees on the cost of every bundle, the problem degenerates to load balancing, and the potential $\phi_i = c(A_i)$ telescopes *exactly* rather than merely bounding the arcs. That exactness is what makes the proof three lines long, and it is precisely what fails as soon as two agents disagree.

For $n = 2$ the conjecture holds as well, with total subsidy at most 1, but here the theorem is somebody else's: by Remark 9 two-agent chore division *is* two-agent goods division with the bundles swapped, so the content is [BKNS22, Theorem 4] and ours is only the reduction. As noted there, that reduction consumes the identity $M \setminus A_j = A_i$ and so says nothing for $n \ge 3$.

### 5.3 Small bundles, and fewer chores than agents

Every other case in this section, and every approach in the paper, eventually needs an *exchange* step: given a bad allocation, produce a better one by moving items. That is exactly what dichotomous costs refuse to supply, since a bundle of positive cost need not contain any single item whose removal lowers it — for instance $c(S) = \min(\max(0, |S| - 1), 1)$ on a three-element set has $c = 1$ while every single removal leaves it at 1. The following theorem needs no exchange at all; it reads the conclusion straight off the cycle-closing bound.

**Theorem 22** (Small-bundle theorem)**.** *Let $\mathcal{A}$ be envy-freeable and suppose every bundle costs at most one unit to every agent: $c_v(A_i) \le 1$ for all $v, i \in N$. Then $p^* \in \{0,1\}^n$ and the total subsidy is at most $n - 1$.*

**Proof** By Theorem 28, $\ell_{\mathcal{A}}(u) \le \max\left(0, \max_{v \ne u}[c_v(A_u) - c_v(A_v)]\right) \le \max(0, 1 - 0) = 1$, using $c_v(A_u) \le 1$ and $c_v(A_v) \ge 0$. Observation 12 then gives $p^* \in \{0,1\}^n$, and the endpoint of a heaviest path has $\ell = 0$, so the total is at most $n - 1$. □

**Corollary 23** (Fewer chores than agents)**.** *If $m \le n$ then Conjecture 2 holds, and a witness is computable by a single minimum-cost bipartite matching.*

**Proof** Among all allocations giving each agent at most one chore, take one minimising $\sum_i c_i(A_i)$; such an allocation exists because $m \le n$, and it is a minimum-cost matching between agents and chores. Every permutation of its own bundles is again an allocation of this form, so the minimisation was over a set containing all of them, and condition (ii) of Theorem 10 holds: $\mathcal{A}$ is envy-freeable. Each bundle has $|A_i| \le 1$, so $c_v(A_i) \le |A_i| \le 1$ for every $v$ — because marginals lie in $\{0, 1\}$ — and Theorem 22 applies. The bound is tight by Example 1, which has $m = n - 1$. □

The hypothesis of Theorem 22 is genuinely weaker than $m \le n$: it also covers instances with many chores in which every agent finds all but a few of them free, so that no bundle is expensive to anybody. Both statements were checked semantically against the true longest-path computation — exhaustively for $m \le 3$, $n \le 3$, on 3,000 random instances for the theorem, and on 111,232 instances for the corollary — with no violations (`structural.py`).

**Remark 24** (Why this is worth stating despite being easy)**.** The proof is three lines and the class is small, so the content is not the theorem but the *mechanism*: it is the only argument in this paper that reaches $\ell \le 1$ without an exchange step, a balance hypothesis, a schedule, or a search. It does so by making the cycle-closing bound tight from the far side — bounding $c_v(A_u)$ from above rather than $c_v(A_v)$ from below. Whether that trick has a wider reach is open; the obvious next question is what happens when bundles cost at most 2, where the same computation gives only $\ell \le 2$ and the argument stops.

### 5.4 Instances admitting a uniformly balanced family

The fourth case is not a restriction on the valuation class but on the instance, and it is the only one of the four that is genuinely new here.

**Theorem 25.** *Conjecture 2 holds for every instance admitting a uniformly balanced family, i.e. a partition of $M$ into $n$ bundles that every agent values within one unit of each other (Theorem 64). Given such a family, an envy-free solution with $p^* \in \{0,1\}^n$ and total subsidy at most $n-1$ is obtained by assigning the bundles by a single maximum-weight matching.*

**Proof** Proposition 65. □

Three things should be said about the scope of this, none of which the proof makes evident.

- **It is a certificate, not an algorithm.** Verifying a proposed family and assigning it are polynomial; *finding* one is a search over partitions, and no polynomial procedure for that is known. So Theorem 25 decides an instance only once somebody supplies the family.
- **The class is large but not everything.** It contains every instance at $n = m = 3$ — all 9,880, checked exhaustively — and every hard instance the project had accumulated before it was tested. It does *not* contain the discrepancy instance of Proposition 66, at $n = 3$ and $m = 4$, nor the certified-hard set-splitting instance.
- **Membership is not the same as difficulty.** The instances outside the class are not the instances that need large subsidies; on the counterexample of Proposition 66, 19 allocations are good and several need no subsidy at all. Uniform balance is a property of the certificate, not of the instance's hardness.

So this is a solved case in the same sense as the others in this section: real, checkable, and not load-bearing. What it contributes is the counterexample and Remark 68, which say that any complete method must reason about paths rather than arcs.

# 6 Approach 1: Item-by-Item Insertion

The algorithm of [BKNS22] maintains an envy-free solution $(\mathcal{A}^t, p^t)$ over a partial allocation with $p^t \in \{0,1\}^n$, and at each step allocates one further good, repairing the invariant by permuting whole bundles among the agents when necessary. Nothing already allocated is ever moved between bundles. This section follows that template on the chore side. Two of its steps come out *easier* than in [BKNS22]; the template nevertheless cannot be completed, and Section 6.3 proves it.

Throughout, $\mathcal{A}$ is a partial allocation, $g \notin \bigcup_i A_i$ is an unallocated chore, and $Z^x$, $\delta_x$, $\delta_{i,x}$ are as in Section 3.1.

## 6.1 The easy case is easier than for goods

**Theorem 26** (Free insertion)**.** *Let $\mathcal{A}$ be envy-freeable and suppose $\delta_x = 0$ for some agent $x$, that is, $c_x(A_x \cup \{g\}) = c_x(A_x)$. Then $Z^x$ is envy-freeable and $\ell_{Z^x}(i) \le \ell_{\mathcal{A}}(i)$ for every $i$.*

**Proof** By Lemma 13, arcs between agents other than $x$ are unchanged; arcs out of $x$ satisfy $w_{Z^x}(x,j) = w_{\mathcal{A}}(x,j) + \delta_x = w_{\mathcal{A}}(x,j)$; and arcs into $x$ satisfy $w_{Z^x}(i,x) = w_{\mathcal{A}}(i,x) - \delta_{i,x} \le w_{\mathcal{A}}(i,x)$. So every arc weight weakly decreases. No positive-weight cycle can therefore appear, and no path weight can rise; the claims follow from Theorems 10 and 11. □

It is worth recording what is *absent* from that proof. There is no hypothesis that $x$ be maximally subsidised, no permutation of bundles, and no extendability machinery: the corresponding step of [BKNS22] needs all three, because for goods the insertion raises the arcs *into* the receiver and those have to be controlled by placing her in $\mathrm{M}(p)$. Here the insertion lowers them for free. Consequently:

**Observation 27.** *The only case requiring work is a chore whose marginal cost is* 1 *for every agent on her own current bundle: $\delta_x = 1$ for all $x \in N$. We call this the* hard case.

### 6.2 A cycle-closing bound

The following is the cheapest general handle on $\ell_{\mathcal{A}}$ available, and it is the chore form of the device [BDN+20] uses in its Section 4, where a lower bound of $-1$ on every arc weight is shown to force a subsidy of at most one per agent.

**Theorem 28** (Cycle-closing bound)**.** *For any envy-freeable allocation $\mathcal{A}$ and any agent $u$,*

$$\ell_{\mathcal{A}}(u) \;\le\; \max\Big(0,\; \max_{v \neq u} \big[\, c_v(A_u) - c_v(A_v) \,\big]\Big).$$

**Proof** The empty path contributes 0. Let $P$ be a nonempty path from $u$ to some $v \neq u$. Appending the arc $(v,u)$ closes $P$ into a directed cycle, whose weight is at most 0 because $\mathcal{A}$ is envy-freeable (Theorem 10). Hence $w_{\mathcal{A}}(P) \le -w_{\mathcal{A}}(v,u) = c_v(A_u) - c_v(A_v)$. □

**Corollary 29.** *If $c_v(A_u) \le c_v(A_v) + 1$ for all $u, v \in N$ — no agent would suffer more than one extra unit by taking anybody else's bundle — then $p^* \in \{0,1\}^n$.*

The hypothesis is sufficient and far from necessary: an agent with $c_i \equiv 0$ can absorb everything, which breaks the hypothesis while the subsidy stays 0. It is nevertheless what does the work in Theorem 18, where the telescoping bound $w_{\mathcal{A}}(i,j) \le u_i - u_j$ supplies exactly this.

### 6.3 The obstruction

In the hard case, Lemma 13 says every arc leaving the receiving agent $x$ rises by exactly 1. Since a path starting at $x$ then rises by 1, the condition $p_x = 0$ is *necessary*. The following theorem shows that no condition of this kind can be sufficient.

Say that $(\mathcal{A}, p)$ is *extendable with $g$* if there is a permutation $\sigma$ such that $\mathcal{A}_\sigma$ is envy-freeable and some agent $\kappa$ has $c_\kappa(A_{\sigma(\kappa)} \cup \{g\}) = c_\kappa(A_{\sigma(\kappa)})$ — that is, if after some welfare-preserving reassignment of the existing bundles, somebody can take $g$ for free and Theorem 26 applies.

**Theorem 30** (The insertion template cannot be completed)**.** *There is a three-agent instance with negative dichotomous valuations, an envy-freeable partial allocation $\mathcal{A}$ with $p^* \in \{0,1\}^3$, and an unallocated chore $g$, such that for* every *agent $x \in N$ the allocation $Z^x$ requires a subsidy of* 2 *— even though the full instance admits an allocation with subsidy* 0*.*

**Proof** Take $M = \{a_1, a_2, g\}$ and

$$c_1(S) = \max\big(0, |S| - 1\big), \qquad c_2(S) = c_3(S) = |S|\,.$$

Each is dichotomous: $c_1$ has marginals $0, 1, 1$ as $|S|$ runs $0, 1, 2$, and $c_2, c_3$ are binary additive. Note that $c_1$ is *supermodular*, which is legal — neither [BKNS22] nor the present paper restricts to submodular valuations — and the instance genuinely needs it.

Consider the partial allocation $\mathcal{A} = (\{a_1, a_2\}, \emptyset, \emptyset)$. Its envy graph has weight matrix

$$\big(w_{\mathcal{A}}(i,j)\big)_{i,j} = \begin{pmatrix} 0 & 1 & 1 \\ -2 & 0 & 0 \\ -2 & 0 & 0 \end{pmatrix},$$

which has no positive-weight cycle, and $p^* = (1, 0, 0)$. So $(\mathcal{A}, p^*)$ is a legal state of the invariant.

The marginal cost of $g$ is 1 for every agent on her own bundle: $c_1(\{a_1, a_2, g\}) - c_1(\{a_1, a_2\}) = 2 - 1 = 1$ and $c_i(\{g\}) - c_i(\emptyset) = 1$ for $i = 2, 3$. So we are in the hard case. Moreover $(\mathcal{A}, p^*)$ is not extendable with $g$: agent 1 must keep $\{a_1, a_2\}$ under any envy-freeable reassignment, since that bundle costs her 1 but costs agents 2 and 3 two apiece, and no reassignment gives anybody a bundle on which $g$ is free.

Inserting $g$ into each bundle in turn gives the minimum subsidy vectors

$$Z^1 \mapsto (2, 0, 0), \qquad Z^2 \mapsto (2, 1, 0), \qquad Z^3 \mapsto (2, 0, 1),$$

each of which has an entry equal to 2. Yet the complete allocation $(\{a_1\}, \{a_2\}, \{g\})$ has an identically zero envy matrix and subsidy $(0, 0, 0)$. □

The failure is not that envy-freeability breaks — it does not.

**Lemma 31.** *In the hard case, if $(\mathcal{A}, p^*)$ is not extendable with $g$, then $Z^x$ is envy-freeable for* every $x \in N$.

**Proof** Cycles avoiding $x$ are unchanged by Lemma 13. Let $C$ be a cycle through $x$ and let $i$ be its predecessor of $x$. The only arcs of $C$ that change are the one leaving $x$, which rises by $\delta_x = 1$, and the one entering $x$ from $i$, which falls by $\delta_{i,x}$; hence $w_{Z^x}(C) = w_{\mathcal{A}}(C) + 1 - \delta_{i,x}$.

If $w_{\mathcal{A}}(C) \le -1$ then $w_{Z^x}(C) \le 0$ immediately. If $w_{\mathcal{A}}(C) = 0$, rotating the bundles along $C$ is a reassignment that preserves total cost, so by Theorem 10(ii) the rotated allocation is again envy-freeable, and in it agent $i$ receives $A_x$. Non-extendability therefore forces $c_i(A_x \cup \{g\}) \neq c_i(A_x)$, i.e. $\delta_{i,x} = 1$, and again $w_{Z^x}(C) \le 0$. Since $\mathcal{A}$ is envy-freeable no cycle has $w_{\mathcal{A}}(C) \ge 1$, so these cases are exhaustive. □

So envy-freeability survives every insertion; it is only the $\{0, 1\}$ bound on the subsidy that breaks. The reason is visible in Section 3.1. A path $u \rightsquigarrow i \to x \to y \rightsquigarrow$ of weight 1 that merely *passes through* $x$ has its outgoing arc raised by $\delta_x = 1$ and its incoming arc lowered by $\delta_{i,x}$, so it becomes a path of weight 2 whenever $\delta_{i,x} = 0$. No condition on $p_x$ prevents this. In the instance above both $x = 2$ and $x = 3$ are hit this way by $i = 1$.

Nor can the repair subroutine of [BKNS22] be mirrored. Its navigation rule walks to an agent that has just required a subsidy of at least 2; in the goods world such an agent

is automatically maximally subsidised, which is the property the argument needs. In the chore world the same agent automatically has $p_u = 1$, and is therefore precisely the kind of agent to whom the chore must *not* be given.

**Remark 32** (What this rules out)**.** Any proof of Conjecture 2 must be allowed to **re-open already-allocated bundles**. The template of [BKNS22] — allocate one item at a time, never move an item that has been allocated, and repair only by permuting whole bundles — is provably insufficient, by Theorem 30. This is a genuine asymmetry between the goods and chore problems and not a gap in the present analysis, so effort spent strengthening the insertion invariant is effort wasted.

# 7 Approach 2: Selecting a Utilitarian-Optimal Allocation

Theorem 30 rules out building the allocation one chore at a time. The natural response is to stop building it at all and instead *select* a good allocation globally, from a set small enough to reason about and rich enough to contain a witness. Utilitarian optimality is the obvious candidate for two reasons. First, by Theorem 10 an allocation minimising $\sum_i c_i(A_i)$ over *all* allocations, and not merely over reassignments of its own bundles, is automatically envy-freeable, so half of what we need is free. Second, it carries a strong exchange property: if $g \in A_i$ is *active* for $i$, meaning $c_i(A_i) - c_i(A_i \setminus \{g\}) = 1$, then $c_j(A_j \cup \{g\}) = c_j(A_j) + 1$ for every $j$ — nobody can absorb it for free, or the allocation would not have been optimal. Combined with Theorem 28, a path of weight at least 2 from $u$ to $v$ forces $c_v(A_u) \geq c_v(A_v) + 2$, so $A_u$ carries at least two units that are live for $v$, and one would like to transfer one of them along the path and show a potential drops.

This is what Conjecture 17 proposes. It is false.

## 7.1 Utilitarian optimality is the wrong restriction

**Proposition 33.** *Conjecture 17 is false. There is an instance with $n = 3$ agents and $m = 4$ chores whose* unique *utilitarian-optimal allocation requires a subsidy of* 2*, although another allocation — of strictly greater total cost — admits a subsidy of* 0*.*

**Proof** Take $M = \{g_1, g_2, g_3, g_4\}$ and the costs tabulated below.

| $S$ | $c_1$ | $c_2$ | $c_3$ | $S$ | $c_1$ | $c_2$ | $c_3$ |
|---|---|---|---|---|---|---|---|
| $\emptyset$ | 0 | 0 | 0 | $g_3g_4$ | 2 | 1 | 1 |
| $g_1$ | 1 | 1 | 1 | $g_1g_2g_3$ | 2 | 3 | 2 |
| $g_2$ | 1 | 1 | 1 | $g_1g_2g_4$ | 3 | 2 | 2 |
| $g_3$ | 1 | 1 | 0 | $g_1g_3g_4$ | 2 | 2 | 2 |
| $g_4$ | 1 | 1 | 1 | $g_2g_3g_4$ | 2 | 2 | 2 |
| $g_1g_2$ | 2 | 2 | 2 | $g_1g_2g_3g_4$ | 3 | 3 | 3 |
| $g_1g_3$ | 1 | 2 | 1 | | | | |
| $g_1g_4$ | 2 | 2 | 2 | | | | |
| $g_2g_3$ | 1 | 2 | 1 | | | | |
| $g_2g_4$ | 2 | 2 | 2 | | | | |

Each column is 0 at $\emptyset$ and has every marginal in $\{0,1\}$, so all three are negative dichotomous in cost form.

*The utilitarian optimum.* Agent 1 pays 1 for every singleton, so $c_1(A_1) = 0$ only if $A_1 = \emptyset$; agent 3 pays 0 for $\{g_3\}$ but 1 for every superset, so $c_3(A_3) = 0$ only if $A_3 \subseteq \{g_3\}$. Any allocation of total cost below 2 would need at least two agents at cost 0, forcing $A_1 = \emptyset$ and $A_3 \subseteq \{g_3\}$ and hence $A_2 \supseteq \{g_1, g_2, g_4\}$, which costs agent 2 at least 2. So the minimum total cost is 2, attained only by

$$\mathcal{A}^\star \;=\; \big(\emptyset,\ \{g_1, g_2, g_4\}\,,\ \{g_3\}\,\big), \qquad 0 + 2 + 0 = 2.$$

Its envy graph has $w_{\mathcal{A}^\star}(2,1) = 2 - 0 = 2$, so $\ell_{\mathcal{A}^\star}(2) \geq 2$; in fact the minimum subsidy is $p^* = (0, 2, 0)$.

*A zero-subsidy allocation.* Take $\mathcal{B} = (\{g_1, g_3\}\,,\ \{g_2\}\,,\ \{g_4\})$, of total cost $1 + 1 + 1 = 3$. Its envy graph has weights

$$\big(w_{\mathcal{B}}(i,j)\big)_{i,j} = \begin{pmatrix} 0 & 0 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix},$$

every entry of which is at most 0. Hence there is no positive-weight cycle, every directed path has weight at most 0, and $p^* = (0, 0, 0)$. □

Two features make this sharper than a bare counterexample. The utilitarian-optimal allocation is *unique*, so there is no tie to break and no selection rule inside the optimal set can rescue the conjecture. And the gap is the largest possible in one step: subsidy 2 against subsidy 0, at a cost of one extra unit of total burden. Restricting attention to utilitarian-optimal allocations is therefore not a neutral simplification but an actively wrong one.

**Remark 34** (What this rules out)**.** Conjecture 2, if true, is *not* witnessed by welfare-maximising allocations, and the programme of finding a tie-breaking rule inside the utilitarian-optimal set is unsound as posed — on this instance the set is a singleton and the witness lies outside it. The exchange property quoted at the head of this section remains true and remains usable; what fails is the assumption that a witness may be sought among cost-minimising allocations at all. Any search procedure must be free to give up total cost.

The consequence is worth stating in the strongest available form, because refinement schemes are easy to propose and all of them die here at once.

**Corollary 35** (No refinement of the utilitarian-optimal set can work)**.** *Let $\mathcal{R}$ be any rule selecting a nonempty subset of the utilitarian-optimal allocations. Then $\mathcal{R}$ fails on the instance of Proposition 33: every allocation it can return requires a subsidy of* 2*, while some allocation outside the utilitarian-optimal set requires none.*

**Proof** On that instance the utilitarian-optimal set is the singleton $\{\mathcal{A}^\star\}$, so $\mathcal{R}$ returns $\{\mathcal{A}^\star\}$, and $\ell_{\mathcal{A}^\star}(2) = 2$. □

**Remark 36** (A tempting refinement, and why it dies)**.** Three independent signals — Theorem 15, the balancedness guarantee of [BDN+20], and Remark 48 — point at *cardinality*

*balance* as the missing ingredient, which suggests refining the utilitarian-optimal set lexicographically by the descending-sorted cardinality vector and then by the descending-sorted own-cost vector. Randomised search over roughly 1050 instances at $n \in \{3, 4\}$ and $m \leq 5$ finds no failure, and neither refinement alone survives that same test, so the pair looks strong. It is nevertheless false, by Corollary 35: on the instance of Proposition 33 both refinements are no-ops on a singleton, and the selected allocation needs subsidy 2 against 0 achievable elsewhere. Verified directly by `checkD.py`.

The episode is worth recording for two reasons. First, it is the sampler bias of Section 12.1 firing again: the failure occurs in roughly one instance in $2.6 \times 10^4$, so a sweep of $10^3$ was two orders of magnitude short of being able to see it. Second, the slip underneath is a subtle misreading of Theorem 10. That theorem says an envy-freeable allocation maximises welfare across reassignments of *its own bundles*; it does *not* say that an envy-freeable allocation is globally welfare-maximising. Restricting to utilitarian-optimal allocations is therefore not free, and the zero-subsidy allocation of Proposition 33 — of total cost 3 against the optimum 2, and envy-freeable — is a direct witness that the two notions differ.

### 7.2 Selection rules that fail

Before Proposition 33 was found, three selection rules inside the utilitarian-optimal set were tested, and all three fail on instances where some optimal allocation *does* work. They are recorded because each is a natural first guess.

- **Minimising total perceived load** $\Psi(\mathcal{A}) := \sum_i \sum_j c_j(A_i)$, which prefers bundles that are cheap in everybody's opinion rather than only in their holder's.
- **Leximin on the perceived-load vector** $\left(\max_j c_j(A_i)\right)_{i \in N}$, which tries to prevent any single bundle from looking heavy to anyone.
- **Local search by single chore transfers** on the objective $\left(\max_i \ell_{\mathcal{A}}(i), \sum_i \ell_{\mathcal{A}}(i)\right)$ restricted to the utilitarian-optimal set. This has bad local minima: there are optimal allocations with subsidy 2 from which no single transfer that stays optimal improves the objective.

The local-search failure is the informative one. Single transfers are too coarse a move set, because the utilitarian-optimal allocations of a dichotomous instance are the bases of a structure whose natural moves are exchange *paths and cycles*, not single steps — the same phenomenon that makes Yankee Swap and matroid union the right tools in the matroid-rank literature [BEF21]. A move set matched to that structure is the obvious thing to try next, and remains untried: exchange-path local search over the full allocation lattice — not restricted to the utilitarian-optimal set, per Remark 34 — with the counterexample hunt re-run under it.

## 8 Approach 3: The Replica Transform and the Peel Reformulation

Both preceding approaches fail because of *when* decisions are made. Item-by-item insertion (Section 6) commits a chore to an owner and cannot take it back; selecting a utilitarian optimum (Section 7) commits to the wrong allocation outright. This section changes

coordinates so that the commitment is deferred. The chore problem is re-expressed as a *goods* problem on a replicated item set, and then read dynamically as a process that relieves agents of chores one at a time. Ownership of a chore is decided only by the last move that touches it.

The payoff is that the proof orientation of [BKNS22] becomes correct again and the obstruction of Theorem 30 ceases to bind. The price is a new constraint — coverage — which is then shown to be non-vacuous.

### 8.1 The replica instance

Fix a negative dichotomous instance with cost functions $c_i$ on chores $M = \{a_1, \dots, a_m\}$. Build a *replica instance* $\hat{I}$ whose item set carries $n-1$ copies of a good $b_j$ for each chore $a_j$:

$$\hat{M} \;:=\; \left\{ b_j^{(1)}, \dots, b_j^{(n-1)} \;:\; j \in M \right\}.$$

For $S \subseteq \hat{M}$ let $\tau(S) \subseteq M$ be the set of *types* occurring in $S$, and define

$$\hat{v}_i(S) \;:=\; c_i(M) \;-\; c_i\big(M \setminus \tau(S)\big).$$

The reading is that *handing agent $i$ a copy of $b_j$ relieves $i$ of chore $a_j$*. Each chore ends with exactly one owner, so exactly $n-1$ agents must be relieved of it — hence $n-1$ copies — and no agent may be relieved of the same chore twice, so no agent may hold two copies of one type. That last clause is load-bearing rather than cosmetic: it is what makes $\hat{v}_i$ well defined as a dual, since a second copy of a type has marginal 0 and never +1 again.

**Remark 37** (The naive reading is false)**.** If one instead declares a copy of $b_j$ to be worth +1 to whoever receives it, the construction collapses. An agent indifferent to $a_j$ would gain from $b_j$, every $\hat{v}_i$ would reduce to $|\tau(S)|$, and the induced envy graph would be $\hat{w}(i,k) = |B_i| - |B_k|$ — the pure cardinality term already isolated in Theorem 15, carrying none of the cost information. The correct reading is *marginal*: a copy of $b_j$ is worth to agent $i$ exactly $i$'s marginal cost of $a_j$ on what remains of $i$'s workload.

This also completes Remark 9. There the dual $\tilde{v}_i(S) = c_i(M) - c_i(M \setminus S)$ settled $n = 2$ and broke at $n \geq 3$ because $M \setminus A_i$ stops being a bundle of the partition. Replication is precisely the repair: with $n-1$ copies of every type, the $n$ complements $M \setminus A_1, \dots, M \setminus A_n$ become *simultaneously* realisable as disjoint bundles.

### 8.2 The transform

Call an allocation $B = (B_1, \dots, B_n)$ of all of $\hat{M}$ a *coverage allocation* if no $B_i$ contains two copies of the same type.

**Theorem 38** (Replica transform)**.** *Let $c_1, \dots, c_n$ be* dichotomous *in the sense of Theorem 7 — that is, every marginal $c_i(S \cup \{g\}) - c_i(S)$ lies in $\{0, 1\}$, which already forces $c_i$ to be monotone. Then*

*(i) each $\hat{v}_i$ is a dichotomous valuation on $\hat{M}$ in the sense of [BKNS22];*

*(ii)* $\Phi : \mathcal{A} \mapsto B$, *where* $B_i$ *receives one copy of each type in* $M \setminus A_i$, *is a bijection between partitions of* $M$ *and coverage allocations of* $\hat{M}$, *up to relabelling copies of a type;*

*(iii) for corresponding* $\mathcal{A}$ *and* $B$ *the envy graphs agree arc for arc:* $\hat{w}_B(i,k) = w_{\mathcal{A}}(i,k)$ *for all* $i, k$.

**Proof** *(i)* Normalisation: $\hat{v}_i(\emptyset) = c_i(M) - c_i(M) = 0$. Monotonicity: $S \subseteq S'$ gives $\tau(S) \subseteq \tau(S')$, hence $M \setminus \tau(S) \supseteq M \setminus \tau(S')$, and $c_i$ is monotone, so $\hat{v}_i(S) \leq \hat{v}_i(S')$. Marginals: take $b \notin S$ of type $j$. If $j \in \tau(S)$ then $\tau(S \cup \{b\}) = \tau(S)$ and the marginal is 0. Otherwise, writing $T := M \setminus \tau(S)$, we have $M \setminus \tau(S \cup \{b\}) = T \setminus \{j\}$ with $j \in T$, so

$$\hat{v}_i(S \cup \{b\}) - \hat{v}_i(S) \;=\; c_i(T) - c_i(T \setminus \{j\}) \;\in\; \{0, 1\}\,,$$

because $c_i$ has marginals in $\{0, 1\}$.

*(ii)* Forward: type $j$ is absent from exactly one bundle of $\mathcal{A}$, its owner's, so exactly $n-1$ agents require a copy of $b_j$ and the $n-1$ available copies are routed injectively. Which physical copy goes where is immaterial, since $\hat{v}_i$ depends on $S$ only through $\tau(S)$. By construction no $B_i$ holds a duplicate and $\bigcup_i B_i = \hat{M}$. Backward: in a coverage allocation the $n-1$ copies of type $j$ lie in $n-1$ *distinct* bundles, so exactly one agent's bundle contains no copy of $j$; setting $\Psi(B)_i := M \setminus \tau(B_i)$ therefore puts $j$ in exactly one $\Psi(B)_i$, making $\Psi(B)$ a partition. Finally $\Psi(\Phi(\mathcal{A}))_i = M \setminus (M \setminus A_i) = A_i$, and $\Phi(\Psi(B))$ agrees with $B$ up to which copy of a type sits where.

*(iii)* For corresponding $\mathcal{A}, B$ we have $\tau(B_k) = M \setminus A_k$, so $\hat{v}_i(B_k) = c_i(M) - c_i(A_k)$ and

$$\hat{w}_B(i,k) = \hat{v}_i(B_k) - \hat{v}_i(B_i) = \big(c_i(M) - c_i(A_k)\big) - \big(c_i(M) - c_i(A_i)\big) = c_i(A_i) - c_i(A_k) = w_{\mathcal{A}}(i,k). \quad \square$$

**Remark 39** (Monotonicity alone is not enough, and which clause does what)**.** The two halves of the hypothesis do different jobs in the proof of (i), and it is worth separating them because the statement is easy to misread. Monotonicity of $c_i$ is what makes $\hat{v}_i$ monotone, and *nothing else*. The binary-marginal condition is what makes $\hat{v}_i$'s marginals binary, because those marginals are *downward* marginals of $c_i$:

$$\hat{v}_i(S \cup \{b\}) - \hat{v}_i(S) \;=\; c_i(T) - c_i(T \setminus \{j\}), \qquad T = M \setminus \tau(S),$$

so they inherit exactly the range of $c_i$'s own marginals. Monotonicity alone bounds this only by $\geq 0$.

The failure is real rather than an artefact of the proof. Take $m = 2$ with $c(\emptyset) = c(\{a\}) = c(\{b\}) = 0$ and $c(\{a, b\}) = 2$. This $c$ is monotone and normalised but has a marginal of 2, and the resulting dual has $\hat{v}(\{b_a\}) - \hat{v}(\emptyset) = 2 - 0 = 2$, so $\hat{v}$ is not dichotomous and [BKNS22] does not apply to $\hat{I}$ at all. Conversely, the normalisation $c_i(\emptyset) = 0$ is *not* used in Theorem 38: $\hat{v}_i(\emptyset) = c_i(M) - c_i(M) = 0$ regardless, and the constant $c_i(M)$ cancels in part (iii). It is retained only because it is part of the model. Verified by `monotone_check.py`, which also confirms by exhaustive enumeration that binary marginals imply monotonicity, so the operative hypothesis is a single clause.

Part (i) is worth stating on its own: *the dual of a dichotomous cost function is a dichotomous value function*, so the class is closed under this duality and the hypotheses of [BKNS22] apply to $\hat{I}$ verbatim, with no additivity, submodularity or subadditivity assumed anywhere. Because the graphs of part (iii) are equal, envy-freeability, every $\ell(i)$, the whole set of feasible subsidy vectors and the $\{0, 1\}$ target all transfer in both directions without loss.

**Corollary 40** (Coverage is the entire gap)**.** *Conjecture 2 holds if and only if every replica instance $\hat{I}$ admits a* coverage *allocation $B$ with $\ell_B(i) \le 1$ for all $i$.*

Applying [BKNS22] to $\hat{I}$ as a black box is legal by Theorem 38(i) and returns some $B$ with $\hat{p} \in \{0,1\}^n$ and total at most $n-1$. But it is free to place two copies of one type in one bundle. If it does, that type is absent from at least two bundles, the chore has at least two owners, and $\Psi(B)$ is not a partition. Coverage is the sole residual difficulty — and, by Section 8.5, a real one.

**Lemma 41** (Duplicate extraction is envy-neutral)**.** *Removing from $B_x$ a copy whose type occurs at least twice in $B_x$ leaves $\tau(B_x)$ unchanged, hence leaves every $\hat{v}_k(B_x)$ unchanged, hence leaves the entire envy graph unchanged.*

**Proof** Immediate, since $\hat{v}_k(S)$ depends on $S$ only through $\tau(S)$. □

So any output of [BKNS22] may be stripped to a duplicate-free partial allocation for free. The difficulty is never in removing duplicates; it is in *re-inserting* the stripped copies into agents that do not already hold that type.

### 8.3 The peel process

Read a coverage allocation as being built one copy at a time and translate back into chores.

**Definition 42** (Peel process)**.** *A* state *is a* workload profile $W = (W_1, \dots, W_n)$ *with $W_i \subseteq M$, such that the* owner-candidate set $S_j := \{i : j \in W_i\}$ *is nonempty for every $j \in M$. Read $W_i$ as "agent $i$ is still on the hook for $W_i$".*

- ***Root:*** *$W_i = M$ for every $i$.*
- ***Peel*** peel$(x, j)$*: legal iff $j \in W_x$ and $|S_j| \ge 2$; it sets $W_x \leftarrow W_x \setminus \{j\}$.*
- ***Permutation:*** *replace $W$ by $(W_{\sigma(1)}, \dots, W_{\sigma(n)})$.*
- ***Terminal:*** *$|S_j| = 1$ for every $j$; the unique element of $S_j$ is the* survivor*, i.e. the owner of $j$, and the induced allocation is $A_i = \{j : S_j = \{i\}\}$.*
- ***Graph:*** *$w_W(i,k) = c_i(W_i) - c_i(W_k)$. The root graph is identically zero.*

*A state is* legal *if its graph has no positive-weight cycle and $\ell_W(i) \le 1$ for every $i$.*

By Theorem 38 a peel state is exactly a duplicate-free partial allocation of $\hat{M}$, a peel is exactly the insertion of one good into one bundle, a permutation is exactly a bundle reassignment, and terminal states are exactly coverage allocations. Nothing is added or lost in the translation.

**Lemma 43** (Peel dynamics)**.** *Write $\mu_k := c_k(W_x) - c_k(W_x \setminus \{j\})$ for $k$'s marginal cost of $j$ on $x$'s residual workload. Then* peel$(x, j)$ *changes only the arcs incident to $x$, by*

$$\Delta w(k,x) = +\mu_k \in \{0,1\} \qquad \text{and} \qquad \Delta w(x,k) = -\mu_x \in \{-1,0\} \qquad (k \ne x).$$

**Proof** Only $W_x$ changes. For $k \ne x$, $w(k,x) = c_k(W_k) - c_k(W_x)$ and $c_k(W_x)$ drops by $\mu_k$, so the arc rises by $\mu_k$. For arcs out of $x$, $w(x,k) = c_x(W_x) - c_x(W_k)$ and $c_x(W_x)$ drops by $\mu_x$. Arcs between two agents other than $x$ involve neither $W_x$ nor $c_x$. □

**The orientation reversal.** This is the point of the whole construction. Compare Lemma 43 against Lemma 13:

| frame | move | in-arcs at $x$ | $x$ should be |
|---|---|---|---|
| goods [BKNS22] | insert a good | rise | maximally subsidised |
| chores, insertion (§6) | insert a chore | fall | unsubsidised, $p_x = 0$ |
| chores, **peel** | relieve of a chore | **rise** | maximally subsidised |

The obstruction of Theorem 30 was exactly this sign clash. The repair subroutine of [BKNS22] navigates to an agent whose tentative subsidy reached 2; in the goods world that agent is automatically maximally subsidised, which is the set the argument needs, whereas in the chore-insertion world she automatically has $p_u = 1$ and is precisely the agent who must *not* receive the chore. In the peel frame the quantity handed out is *relief*, relief correctly flows to the most envious agents, and the orientation points the right way again.

**Lemma 44** (Spectator-free peels are safe)**.** *If $\mu_k = 0$ for every $k \neq x$, then* peel$(x, j)$ *weakly decreases every arc weight, hence $\ell$ pointwise, and preserves envy-freeability — whatever $\mu_x$ may be.*

**Proof** By Lemma 43 the arcs into $x$ are unchanged, the arcs out of $x$ change by $-\mu_x \leq 0$, and all other arcs are untouched. Path and cycle weights are sums of arc weights. □

Lemma 44 is the peel-frame replacement for Theorem 26, and note where the safety condition has moved. In the insertion frame a chore that was free *for the receiver* was unconditionally safe; here what matters is that the chore be free *for everyone else*. That is the goods-side safety structure of [BKNS22], as the identification predicts. The hard case is now a pair $(x, j)$ for which some other agent $k$ has $\mu_k = 1$.

What does *not* transfer is the coverage constraint, which has no analogue in [BKNS22]: any good may go in any bundle there, whereas a peel demands $x \in S_j$ and a type is peelable only while $|S_j| \geq 2$.

### 8.4 The insertion obstruction dissolves

Theorem 30 remains true — it is a theorem about the insertion template, and the peel frame is a different template, not a repair of that one. What follows is that its witness no longer obstructs.

Recall the witness: $M = \{a_1, a_2, g\}$ with $c_1(S) = \max(0, |S| - 1)$ and $c_2 = c_3 = |S|$.

**The trap state is refused before it can form.** The peel-frame counterpart of the trap is $W = (\{a_1, a_2, g\}, \{g\}, \{g\})$, and

$$w_W(1, 2) \;=\; c_1(\{a_1, a_2, g\}) - c_1(\{g\}) \;=\; 2 - 0 \;=\; 2,$$

so $\ell_W(1) = 2$ and the state is already illegal. No legal peel schedule passes through it.

**Remark 45** (The conditioned-remainder principle)**.** The two frames disagree because they compare different things. At a stage where the finished types form a partial assignment $\mathcal{A}^{(t)}$ and the untouched types form $R$, every agent is still on the hook for $R$, so the peel-frame arc is

$$c_i\big(\mathcal{A}_i^{(t)} \cup R\big) - c_i\big(\mathcal{A}_k^{(t)} \cup R\big), \qquad \text{not} \qquad c_i\big(\mathcal{A}_i^{(t)}\big) - c_i\big(\mathcal{A}_k^{(t)}\big).$$

Every comparison is conditioned on the common unfinished remainder. For additive costs the $R$ terms cancel and the two frames coincide — consistent with the additive case already being closed by Theorem 18. For non-additive costs they differ, and on the witness it is exactly the supermodularity of $c_1$ that the conditioning catches: with $g$ still pending, loading agent 1 with both of $a_1, a_2$ already costs 2, and the peel invariant sees it. *The insertion invariant was blind to pending load; the peel invariant has hindsight built in.* This is the substantive content of the change of coordinates.

**A legal schedule reaches the zero-subsidy allocation.** Six peels, every one of them the hard case ($\mu_k = 1$ for all $k$ — this instance never offers a spectator-free move), with the invariant holding throughout. Verified by `peel.py`.

| step | move | resulting $W$ | $\ell$ | $\sum_i \ell(i)$ |
|---|---|---|---|---|
| 0 | root | $(a_1a_2g,\ a_1a_2g,\ a_1a_2g)$ | $(0,0,0)$ | 0 |
| 1 | relieve 1 of $g$ | $(a_1a_2,\ a_1a_2g,\ a_1a_2g)$ | $(0,1,1)$ | 2 |
| 2 | relieve 2 of $g$ [owner$(g) = 3$] | $(a_1a_2,\ a_1a_2,\ a_1a_2g)$ | $(0,0,1)$ | 1 |
| 3 | relieve 3 of $a_1$ | $(a_1a_2,\ a_1a_2,\ a_2g)$ | $(0,0,0)$ | 0 |
| 4 | relieve 1 of $a_2$ | $(a_1,\ a_1a_2,\ a_2g)$ | $(0,1,1)$ | 2 |
| 5 | relieve 2 of $a_1$ [owner$(a_1) = 1$] | $(a_1,\ a_2,\ a_2g)$ | $(0,0,1)$ | 1 |
| 6 | relieve 3 of $a_2$ [owner$(a_2) = 2$] | $(a_1,\ a_2,\ g)$ | $(0,0,0)$ | 0 |

The terminal allocation is $(\{a_1\}, \{a_2\}, \{g\})$, the known zero-subsidy solution. This establishes that the peel template is not killed by the existing obstruction; it does not establish that the template always works, which is Section 8.5. Note also how it meets the requirement of Remark 32 that a correct proof be free to re-open allocated bundles: *peeling never commits a chore's owner until the last peel of that type, so ownership is a deferred decision and there is nothing to re-open.*

### 8.5 The new obstruction: peel dead ends

The natural conjecture is local.

**Conjecture 46** (Local extendability — **refuted**)**.** *From every legal non-terminal state, some legal move exists.*

**Proposition 47.** *Conjecture 46 is false. There are legal, non-terminal peel states from which no sequence of peels and permutations reaches a terminal state without violating the invariant.*

**Proof** A complete backward fixed-point computation over the entire state space of the witness, with both move types and no heuristics, is carried out by `peel3.py`; it reports the following.

| quantity | value |
|---|---|
| states in total ($7^3$) | 343 |
| legal states | 175 |
| terminal (partition) states that are legal | 6 of 27 |
| legal states from which a terminal is reachable within the invariant | 166 |
| **legal dead ends** | **9** |
| root is good | yes |

The smallest dead end is $W = (\{a_1, a_2\}, \{g\}, \{g\})$, which is legal with $\ell = (1, 0, 0)$. The only peelable type is $g$, and both continuations fail: relieving agent 2 or agent 3 of $g$ creates a weight-2 path out of agent 1. Permutations do not help — the profile $(\{g\}, \{a_1, a_2\}, \{g\})$ is legal but its two continuations fail identically. □

**What the dead ends are, in the language of [BKNS22].** This is the sharpest statement available. At $W = (\{a_1, a_2\}, \{g\}, \{g\})$ the remaining unallocated copy is a copy of $b_g$, and it has three possible recipients. Giving it to agent 2 or 3 is a genuine peel and violates the invariant. Giving it to agent 1 — who already holds a copy of $b_g$ — creates a *duplicate*, which is perfectly legal in $\hat{I}$ and, by Lemma 41, changes nothing whatever in the graph. So the algorithm of [BKNS22] never gets stuck on $\hat{I}$: it gets stuck only in the sense that its one legal move is to spend a copy on a duplicate, and spending a copy on a duplicate is exactly the loss of coverage. If agent 1 takes the second $b_g$ then $g$ ends with owner-candidate set $\{2, 3\}$ — two owners, not a partition. Corollary 40 is therefore not a formal restatement; the gap it names is realised.

**Remark 48** (The balance signal — **observation only**)**.** All nine dead ends of the witness have the same shape: *one agent is already committed to a two-element terminal bundle*. They are exactly the three choices of which pair times the three choices of which agent. Correspondingly all six reachable terminals are the six perfect matchings, so on this instance the only legal partitions are the matchings. This is the same signal as Theorem 15 — chores are goods plus cardinality balancing — and as the balancedness guarantee of [BDN+20], arriving from a third direction, and it suggests the missing ingredient is a *balance* discipline rather than a subsidy-based one. Consistent with that, restricting recipients to $\arg\max_i \ell(i)$, the direct mirror of M($p$), shrinks the reachable state space but does *not* avoid the dead ends: a subsidy-based rule alone is not enough. This is a single instance and should not be generalised without more data.

### 8.6 The reduced open problem

**Conjecture 49** (Reachability — **open**)**.** *From the root, some sequence of peels and permutations reaches a terminal state with the invariant holding at every intermediate state.*

Conjecture 49 implies Conjecture 2. The root graph is identically zero, so the invariant holds there; if a legal schedule reaches a terminal state $W^{\text{end}}$ then the induced allocation $\mathcal{A}$ has $G_{\mathcal{A}} = G_{W^{\text{end}}}$, with no positive cycle and $\ell_{\mathcal{A}}(i) \le 1$ for all $i$; Observation 12 then forces $p^* \in \{0, 1\}^n$, and the endpoint of a maximum-weight path has $\ell = 0$, giving a total of at most $n - 1$.

**Remark 50** (The converse is open, and it matters for what the experiments test)**.** Conjecture 49 implies Conjecture 2, but the implication has not been established in the other direction and should not be assumed. Conjecture 2 asserts that a good terminal state *exists*; Conjecture 49 asserts that one is *reachable along a path every state of which is legal*. These can come apart: a good terminal state may exist while every schedule leading to it passes through an illegal intermediate state, and Proposition 47 shows that illegal-looking intermediate states are not hypothetical. The practical consequence is that an instance with a *bad root* would refute Conjecture 49, and hence kill this approach, but would say nothing directly about Conjecture 2, which would have to be attacked by exhaustive search over allocations as in Section 12. Establishing the converse — that a good terminal state, when one exists, is always reachable legally — would make the peel frame a complete reformulation rather than a sufficient one, and is worth attempting on its own.

It is not automatic. Conjecture 49 is a reachability statement about a state space with genuine dead ends, so a proof must supply a *rule* for choosing the next peel that provably never enters the bad region, or else avoid the state space altogether by a potential or exchange argument. Two resources are available that [BKNS22] never had:

1. **Scheduling freedom.** Many types are unfinished at once, and we need only *some* type to admit a legal peel, not a prescribed one.
2. $|S_j| \geq 2$**.** A peelable type always has at least two candidate recipients, so the forbidden recipient set is never all of $N$.

Call a legal non-terminal state a *deadlock* if for every unfinished type $j$ and every permutation, no recipient in $S_j$ preserves the invariant. Proposition 47 shows deadlocks exist; Conjecture 49 asks only that the *root* avoid them. So the target theorem has the shape *there is a peel rule under which no deadlock is ever entered*, and Remark 48 is the first candidate for what that rule must enforce.

### 8.7 The type-ordered restriction

One restriction of the peel process survives every test so far and shrinks the search substantially. Present the chores in *blocks*: decide the owner of $a_1$ completely, then $a_2$, and so on. Coverage is automatic in the peel frame — a peel deletes $j$ from $W_x$, so the same pair $(x, j)$ cannot be peeled twice — and after $r$ complete blocks the state is $W_i = \mathcal{A}_i^{(r)} \cup R$ with $R$ the undecided chores, which is literally the conditioned-remainder principle of Remark 45. A state is then just a triple (partial allocation, current chore, set of agents already relieved of it), which is a far smaller space than the general peel process.

**Conjecture 51** (Type-ordered reachability — **open**)**.** *From the root, some chore order $\pi$, together with some choice of owner and within-block sequence for each chore, reaches a terminal state with the invariant holding at every intermediate state.*

Conjecture 51 implies Conjecture 49 implies Conjecture 2, and it is a strictly smaller search problem than either. The evidence, from `typeorder.py` and `stress.py`, is that the restriction costs nothing: across the witness of Theorem 30 and 890 random dichotomous

instances at $n \in \{3, 4\}$, $m \leq 4$, *every* instance that admits a legal schedule at all admits a type-ordered one. The chore *order* is not free — in 8/120 instances at $n = m = 3$, 16/60 at $n = 3, m = 4$ and 8/60 at $n = 4, m = 3$ only some orders work — but in every instance tested some order does. Note that the schedule exhibited in Section 8.4 is *not* type-ordered, since it interleaves $a_1$ and $a_2$; the table says a type-ordered one exists for that instance anyway, which was not obvious.

What does not survive is the natural recipient rule to pair with it.

**Proposition 52** (arg max recipient selection is insufficient — **refuted**)**.** *Giving each copy to an agent of maximum current subsidy does not suffice, even under the type-ordering.*

**Proof** Take $n = 3$, $m = 2$ with the costs

$$c_1(\emptyset) = 0,\ c_1(\{a_1\}) = c_1(\{a_2\}) = 0,\ c_1(M) = 1; \qquad c_2(S) = \min(|S|, 1); \qquad c_3 = c_1.$$

All three are dichotomous (Theorem 7). Exhaustive search over both chore orders, both roles and all within-block sequences finds no legal type-ordered schedule in which every recipient lies in $\arg\max_i \ell_W(i)$. Dropping only the arg max restriction, the schedule that relieves agent 1 of $a_1$, then agent 3 of $a_1$, then agent 1 of $a_2$, then agent 3 of $a_2$ is legal and terminates at $\mathcal{A} = (\emptyset, \{a_1, a_2\}, \emptyset)$ with $\ell = (0, 1, 0)$. □

This is the signal of Remark 48 again — a subsidy-based recipient rule alone is not enough — now confirmed *inside* the type-ordered restriction, which is the sharper statement. Combined with Theorem 61, which says the recipient in the hard case has zero marginal, the rule provably cannot be read off the subsidy vector: it must see the residual workload, which is exactly what the conditioned-remainder arcs already encode.

### 8.8 A subclass that may fall first — conjectural

On type space the dual of a *supermodular* dichotomous cost is submodular with binary marginals, that is, a matroid rank function; lifting through $\tau$ to the copies is the parallel extension of that matroid, still a matroid rank function. Additive costs dualise to partition-matroid ranks. If that is right — and the parallel-extension step in particular deserves a full proof before being relied on — then supermodular dichotomous chores are exactly matroid-rank goods coverage problems, and the witness of Theorem 30 lies entirely inside this class, since $c_1$ dualises to the rank of $U_{2,3}$, parallel extended.

Matroid rank is where the machinery of the surrounding literature is strongest, through the exchange properties used in [BSY23, BEF21]. That makes Conjecture 2 restricted to supermodular dichotomous costs, attacked by basis exchange in the parallel-copy matroid, the natural intermediate target — sitting between Theorem 18, which is closed, and the full conjecture.

## 9 Approach 4: Encoding Coverage into the Numbers

Corollary 40 reduces Conjecture 2 to a single constraint: the replica instance $\hat{I}$ must admit an allocation in which no agent holds two copies of one type. Applying [BKNS22] to $\hat{I}$ is

legal but returns an allocation free to violate that. The obvious response is to change the numbers so that a violation becomes unattractive — either at the level of the valuations, by charging for duplicates, or at the level of the algorithm, by inflating weights so that the wrong recipient is never selected.

This section closes both. They fail for the same reason, isolated in Section 9.6.

## 9.1 The literal proposal is a no-op

**Theorem 53.** *In $\hat{I}$ the marginal value of a second copy of a type is already exactly 0, for every agent and every set: if $b$ has type $j$ and $j \in \tau(S)$ then $\hat{v}_i(S \cup \{b\}) - \hat{v}_i(S) = 0$.*

**Proof** $\tau(S \cup \{b\}) = \tau(S)$, and $\hat{v}_i$ factors through $\tau$. □

So "let a first copy carry weight and a second carry none" is not a modification of $\hat{I}$; it *is* $\hat{I}$. Remark 37 already records that the other choice, a flat +1 per copy, destroys the transform outright.

The reason this does not help is that zero marginal means *indifferent*, not *averse*. Lemma 41 is the exact statement of the damage: a duplicate is envy-neutral, changing no arc of the envy graph at all. An objective built out of envy comparisons cannot see duplicates, and the dead ends of Proposition 47 exist because of this rather than in spite of it — at the dead end the one legal move is to spend a copy on a duplicate, which is free. To repel a duplicate rather than merely fail to attract it, the duplicate must be made *visible*.

## 9.2 The separation programme, correctly stated

Write $d(S) := |S| - |\tau(S)|$ for the number of duplicated copies in $S$. Call $u = (u_1, \dots, u_n)$ on $\hat{M}$ a *faithful reweighting* if

(D) each $u_i$ is dichotomous — $u_i(\emptyset) = 0$, monotone, all marginals in $\{0, 1\}$ — so that [BKNS22] applies to $(\hat{M}, u)$; and

(F) $u_i(S) = \hat{v}_i(S)$ for every duplicate-free $S$, so that the envy graph of a coverage allocation is unchanged.

Say $u$ *separates* if every non-coverage allocation $B$ has $\max_i \ell_B^u(i) \geq 2$.

**Observation 54** (The programme is logically valid)**.** *If some faithful $u$ separates, then Conjecture 2 holds for that instance.*

**Proof** Applying [BKNS22] to $(\hat{M}, u)$ returns $B$ with $\ell_B^u \in \{0, 1\}^n$; separation forces $B$ to be a coverage allocation; on a coverage allocation (F) gives $u_i(B_k) = \hat{v}_i(B_k)$ for all $i, k$, so $\ell_B^{\hat{v}} = \ell_B^u \leq 1$; and Theorem 38 hands back a chore partition with per-agent subsidy in $\{0, 1\}$ and total at most $n - 1$. □

The idea is therefore not confused — it is a genuine reduction template. The rest of this subsection shows it cannot be filled in.

**Theorem 55** (Duplicate budget)**.** *Every faithful reweighting satisfies $\hat{v}_i(S) \leq u_i(S) \leq \hat{v}_i(S) + d(S)$ for all $i$ and all $S \subseteq \hat{M}$. So the penalty $f_i := u_i - \hat{v}_i$ is non-negative, vanishes on duplicate-free sets, and is bounded by one unit per duplicated copy.*

**Proof** Let $T \subseteq S$ be duplicate-free with $\tau(T) = \tau(S)$, so $|S \setminus T| = d(S)$ and $\hat{v}_i(T) = \hat{v}_i(S)$. Monotonicity gives $u_i(S) \geq u_i(T) = \hat{v}_i(T) = \hat{v}_i(S)$ by (F). Adding the $d(S)$ elements of $S \setminus T$ one at a time raises $u_i$ by at most 1 each by (D), so $u_i(S) \leq u_i(T) + d(S)$. □

The extreme $f_i = d$ is attained, since $u_i = \hat{v}_i + d$ is monotone, normalised and has marginals in $\{0, 1\}$. So *"a duplicate is worth +1 to everybody" is the strongest legal penalty there is*, and any scaling by $\varepsilon \notin \{0, 1\}$ leaves the dichotomous class and forfeits [BKNS22] altogether.

### 9.3 The strongest uniform penalty is a potential

**Theorem 56.** *Let $u_i = \hat{v}_i + \varepsilon d$ for every $i$, with the same $\varepsilon$ and the same $d$ for all agents. Then for every allocation $B$,*

$$w^u_B(i,k) \;=\; w^{\hat{v}}_B(i,k) \;+\; \varepsilon\big(d(B_k) - d(B_i)\big),$$

*and consequently*

*(1) every directed cycle has the same weight in both graphs, so $B$ is envy-freeable under $u$ if and only if it is under $\hat{v}$: a uniform penalty can never* disqualify *an allocation;*

*(2) the agent holding the* most *duplicates has weakly* smaller *subsidy and everyone else weakly larger — the penalty lands on the wrong agents;*

*(3) if $d(B_1) = \cdots = d(B_n)$ the two graphs are identical, so such allocations are invisible to the penalty however large $\varepsilon$ is.*

**Proof** The identity is immediate from the definitions. The correction telescopes to 0 around any cycle, giving (1), and along a path to endpoint minus start, giving (2). For (3) all corrections vanish. □

**Example 57** (Maximally non-coverage, and invisible)**.** Take $n = 3$, $m = 3$ and $c_i(S) = |S|$ for every $i$, and let $B_i$ be both copies of $b_i$. Then $d(B_i) = 1$ for every $i$, so Theorem 56(3) applies; $\hat{v}_i(B_k) = |\tau(B_k)| = 1$ for all $i, k$, every arc is 0 and $\ell^u_B = (0, 0, 0)$. Yet $\Psi(B)_i = M \setminus \{a_i\}$ gives every chore two owners — as far from coverage as it is possible to be. Applying [BKNS22] to this $u$ may return exactly this allocation.

### 9.4 No reweighting whatsoever separates

Theorem 56 leaves open the asymmetric case, in which different agents charge for duplicates of different types. That does break the potential structure and can create positive cycles, so it has to be excluded separately.

**Theorem 58** (No-go)**.** *There is a negative dichotomous instance on which* no *faithful reweighting separates: $n = 3$, $m = 2$, $c_1 = c_2 = c_3 = |\cdot|$.*

**Proof** The replica has types $a_1, a_2$ with two copies each and $\hat{v}_i(S) = |\tau(S)|$. Write $D$ for the pair of copies of $b_1$, and $s_1, s_2$ for the two copies of $b_2$. For $x \in \{1, 2, 3\}$ let $B^{(x)}$ give $D$ to agent $x$ and one of $s_1, s_2$ to each of the others; each is non-coverage.

Let $u$ be faithful. The singletons are duplicate-free, so $u_i(s_1) = u_i(s_2) = 1$ for every $i$ by (F). For $D$ we have $\hat{v}_i(D) = 1$ and $d(D) = 1$, so Theorem 55 gives $u_i(D) = 1 + \delta_i$ with

$\delta_i \in \{0,1\}$ — and $\delta = (\delta_1, \delta_2, \delta_3)$ is the *entire* freedom $u$ has on this family. Fixing $x$ and writing $\{y, z\} = N \setminus \{x\}$, the arcs of $B^{(x)}$ are

$$w(y,x) = \delta_y, \quad w(z,x) = \delta_z, \quad w(x,y) = w(x,z) = -\delta_x, \quad w(y,z) = w(z,y) = 0.$$

If $\delta_x = 1$: every arc out of $x$ is $-1$, every arc into $x$ is at most 1, and the $y \leftrightarrow z$ arcs are 0. Any cycle uses one arc into $x$ and one out, so has weight at most 0; any path out of $y$ has weight at most $\max(0, \delta_y, \delta_z) \le 1$, likewise out of $z$, and $\ell(x) = 0$. So $B^{(x)}$ survives. If $\delta_x = 0$ and $\delta_y = \delta_z = 0$: every arc is 0 and $B^{(x)}$ survives. If $\delta_x = 0$ and $\delta_k = 1$ for some $k \neq x$: the two-cycle $k \to x \to k$ has weight $\delta_k - \delta_x = 1 > 0$, and $B^{(x)}$ is excluded.

So $B^{(x)}$ is excluded exactly when $\delta_x = 0$ and $\delta_k = 1$ for some $k \neq x$. Separation demands all three of $B^{(1)}, B^{(2)}, B^{(3)}$ be excluded, forcing $\delta_1 = \delta_2 = \delta_3 = 0$ and hence no $\delta_k = 1$ — a contradiction. □

**Theorem 59** (Machine-verified strengthening)**.** *On the same instance separation fails even if faithfulness is dropped entirely. Among all* 990 *dichotomous valuations on the four replica copies, none of the* $990^3$ *triples* $(u_1, u_2, u_3)$ *excludes all twelve non-coverage allocations of the shape "one duplicated pair plus two singletons".*

**Proof** Exhaustive, by `dupsep.py`: collapsing the 990 valuations by their behaviour on the critical family leaves 30 classes, and 0 of the resulting class-triples survive the filter. No heuristics are used. The same script also sweeps the separable $P$-family $u_i(S) = \hat{v}_i(S) + \sum_{j \in P_i} (|S_j| - 1)^+$, which for $n = 3$ is the whole separable penalty family, and finds no separating member on $n = 3, m = 2$ (all 64 triples), on $n = 3, m = 3$ with unit costs (all 512), or on the witness of Theorem 30 (all 512). □

The instance of Theorem 58 is *not* a counterexample to Conjecture 2: the coverage allocation $B = (\{b_1^{(1)}, b_2^{(1)}\}, \{b_1^{(2)}\}, \{b_2^{(2)}\})$, i.e. the chore partition $(\emptyset, \{a_2\}, \{a_1\})$, has $\ell = (0,1,1)$ and total $2 = n - 1$. What is refuted is the *method*, on the easiest instance available.

### 9.5 The same failure at the algorithm level

The second form of the idea leaves the valuations alone and instead inflates the weight of the block of copies currently being allocated, intending that receiving a copy drive the recipient's subsidy to 0 so that an agent already holding that type is never chosen again. The diagnosis behind it is right.

**Theorem 60.** *In any run of the algorithm of [BKNS22] on* $\hat{I}$*, the* Extend *branch never places a duplicate. Every coverage violation is created by* FindSink*.*

**Proof** Extend returns a pair $(\rho, k)$ drawn from its loop over $k \in N$ and $\ell \in \mathrm{M}(p)$ satisfying $v_k(A_\ell \cup \{g\}) - v_k(A_\ell) = 1$, and places $g$ into the bundle $B_k = A_\ell$. So the receiving bundle has strictly positive marginal for $g$. By Theorem 53 a copy of $b_j$ has marginal 0 whenever $j \in \tau(S)$, so the receiving bundle contains no copy of $j$. □

But the lever cannot reach the subroutine that does the damage.

**Theorem 61** (FindSink runs only in the zero-marginal regime)**.** *Suppose the algorithm reaches its FindSink branch, i.e.* $(\mathcal{A}, p)$ *is not extendable with* $g$*. Then every* $\ell \in \mathrm{M}(p)$ *satisfies* $v_\ell(A_\ell \cup \{g\}) - v_\ell(A_\ell) = 0$*.*

**Proof** Suppose some $\ell \in \mathrm{M}(p)$ had marginal 1, and run Extend's loop at $k = \ell$, which is then a feasible choice. It computes a maximum-weight matching $\rho$ on the complete bipartite graph between $N \setminus \{\ell\}$ and itself with weights $v_i(A_j)$; maximality against the identity matching gives $\sum_{i \neq \ell} v_i(A_{\rho(i)}) \geq \sum_{i \neq \ell} v_i(A_i)$. Setting $\rho(\ell) = \ell$ adds $v_\ell(A_\ell)$ to both sides, so the welfare test passes and Extend returns $(\rho, \ell)$ — contradicting non-extendability. □

**Corollary 62** (The recipient's subsidy is exactly preserved)**.** *Let* $s$ *be the agent returned by FindSink and* $\mathcal{A}' = (A_1, \ldots, A_s \cup \{g\}, \ldots, A_n)$*. Then* $\ell_{\mathcal{A}'}(s) = \ell_{\mathcal{A}}(s)$*.*

**Proof** [BKNS22] places $s \in \mathrm{M}(p)$, so Theorem 61 gives $v_s(A_s \cup \{g\}) = v_s(A_s)$ and every arc out of $s$ is unchanged. Arcs not incident to $s$ are unchanged, and only arcs into $s$ can rise. Since $\mathcal{A}'$ is envy-freeable it has no positive cycle, so $\ell_{\mathcal{A}'}$ is the maximum weight over *simple* paths; a simple path leaving $s$ never re-enters $s$ and therefore uses only unchanged arcs. □

Three consequences, each fatal on its own.

1. **The lever multiplies a zero.** The premise — that receiving a copy forces the recipient's subsidy to 0 — holds only where the recipient's marginal is positive. Theorem 61 says that in the FindSink branch it is provably 0, and scaling by any weight leaves 0 where it was. Corollary 62 makes it exact: the recipient's subsidy does not move at all. Inflation bites only in the Extend branch, which by Theorem 60 was never broken.

2. **A duplicate is FindSink's safest move.** By Lemma 41 a second copy changes no arc whatsoever, so the subsidy vector after the tentative assignment is literally identical, the subroutine's guard fails on the first test, and it returns immediately. A duplicate is never rejected because there is nothing to detect.

3. **The scaling forfeits the guarantee anyway.** With block weights the marginals lie in $\{0, W_j\}$ rather than $\{0, 1\}$, and the step of [BKNS22] that concludes $p \in \{0, 1\}^n$ from integrality, together with the unit path bounds throughout its analysis, is gone. Using the lever is not *applying* [BKNS22]; it is writing a new algorithm and owing a new proof.

It is worth recording where the lever *does* bite, because that is why it looks like it works. Within a block, once some agent has been relieved of $a_j$ while others still hold it, the arcs into the relieved agent from any holder with marginal 1 carry the inflated weight, so $\mathrm{M}(p)$ collapses onto exactly the holders who care — and those are precisely the agents for whom Extend applies. The lever keeps FindSink from firing at all. It runs out exactly when every remaining holder of $a_j$ has marginal 0: then nothing is inflated, $\mathrm{M}(p)$ reverts to stale values from earlier blocks, and an already-relieved agent can sit at the top of it. The minimal shape is $n = 3$ with a chore of marginal 1 for one agent and 0 for the other two.

### 9.6 Why both fail: holder-blindness

The two failures have one cause, and naming it is the useful output of this section.

A duplicate penalty bites only through the arcs. The arc effect of agent $k$ charging $+1$ for the duplicates in bundle $B_x$ is $+1$ on $w(k, x)$ — but the same charge levied by $x$ on her own bundle is $+1$ on $u_x(B_x)$, which is $-1$ on every arc *out of* $x$. The two cancel around every cycle. So a penalty can disqualify an allocation only if the agent who *holds* the duplicate does not feel it while somebody else does. But $u$ is fixed before the allocation is chosen, and it is the allocation that decides who holds the duplicate. Requiring the holder to be blind for every possible holder forces the penalty to zero, which is exactly the $\delta$ argument of Theorem 58.

Compare the orientation reversal of Section 8.3. There the difficulty was that the navigation rule of [BKNS22] sends the item to the agent who must not receive it. Here the difficulty is that the penalty must be levied on an agent who cannot be identified in advance. Both are the same species of failure: *a rule fixed ex ante against a quantity determined ex post*.

**Remark 63** (What this rules out, and what it does not)**. Ruled out:** encoding coverage into the valuations of the replica instance so that [BKNS22] can be used as a black box. Any such encoding is a dichotomous reweighting, and Theorem 59 kills it. Also ruled out: steering the algorithm of [BKNS22] by inflating weights, by Theorem 61. Together these close the whole family *"encode coverage into the numbers and reuse [BKNS22] as a black box"* from two directions.

**Not ruled out:** coverage-aware *algorithms* — Conjecture 49 and the search for a peel rule that never enters a deadlock. Coverage is a constraint on the allocation, and the right place to enforce a constraint on the allocation is the procedure that builds it, not the objective it is scored against.

## 10 Approach 5: Agent Induction and the Balance Invariant

Every approach so far inducts on *items*. Inducting on *agents* instead is attractive for chores because a newcomer starts with an empty bundle, which is the cheapest bundle there is, so the newcomer is the natural sink and the orientation problem of Section 6 does not arise. This section sets up what such an induction would have to carry, and shows that the natural candidate fails.

### 10.1 What the induction would have to carry

Corollary 29 says that if $\mathcal{A}$ is envy-freeable and every arc satisfies $w_{\mathcal{A}}(i, j) \geq -1$ — equivalently $c_i(A_j) \leq c_i(A_i) + 1$ — then $p^* \in \{0, 1\}^n$. The useful feature of that condition for an induction on agents is that it is *assignment-invariant*: it constrains only the family of bundles, not which agent holds which. That suggests carrying the family rather than the allocation.

**Definition 64** (Uniformly balanced family)**.** *A family of $n$ disjoint bundles covering $M$ is* uniformly balanced *if every agent values all $n$ bundles within one unit of each other:* $\max_t c_i(B_t) - \min_t c_i(B_t) \leq 1$ *for every $i \in N$.*

**Proposition 65.** *If an instance admits a uniformly balanced family, then Conjecture 2 holds for it.*

**Proof** Assign the bundles to agents by a maximum-weight matching. By Theorem 10 the resulting allocation is envy-freeable, and by uniform balance every arc satisfies $w_{\mathcal{A}}(i,j) = c_i(A_i) - c_i(A_j) \geq -1$ — the bound holds for *any* assignment, so in particular for this one. Corollary 29 then gives $p^* \in \{0,1\}^n$, and Observation 12 plus the vanishing of $\ell$ at the endpoint of a heaviest path give a total of at most $n-1$. □

This is exactly the shape an agent induction wants: a purely combinatorial, schedule-free, coverage-free statement, with the step from $n$ to $n+1$ agents being a refinement of the family. It also holds on every hard instance the project has accumulated, and on the full exhaustive family at $n = m = 3$ — all 9,880 instances, with the implication chain of Proposition 65 verified end to end on each (`routeA.py`). It is nevertheless false.

### 10.2 The invariant is unreachable

**Proposition 66.** *There is a negative dichotomous instance with $n = 3$ and $m = 4$, with binary additive costs, admitting no uniformly balanced family.*

**Proof** Take $c_i(S) = |S \cap D_i|$ with

$$D_1 = \{g_1, g_2\}, \qquad D_2 = \{g_1, g_3, g_4\}, \qquad D_3 = \{g_2, g_3, g_4\}.$$

Uniform balance for agent $i$ means the elements of $D_i$ are spread across the three bundles with counts of range at most 1. Three elements over three bundles have range at most 1 only in the pattern $(1,1,1)$, so $D_2$ forces $g_1, g_3, g_4$ into three *different* bundles, one each, and $D_3$ likewise forces $g_2, g_3, g_4$ into three different bundles. Hence $g_2$ must lie in the one bundle containing neither $g_3$ nor $g_4$, which is the bundle containing $g_1$. But two elements over three bundles have range at most 1 only in the pattern $(1,1,0)$, so $D_1$ forces $g_1$ and $g_2$ into *different* bundles. Contradiction. □

The instance is binary additive and therefore already settled by [LMS26]; what it refutes is the method, on a class the literature closes. Indeed 19 of its 81 allocations are good, several with $\ell = (0,0,0)$ — it is an easy instance for the conjecture and an impossible one for the invariant.

The failure is sharper than mere unreachability.

**Proposition 67.** *On the instance of Proposition 66,* every *good allocation has an arc of weight $-2$ or less. Hence no sufficient condition of the form "all arcs $\geq -1$" can certify any of them.*

**Proof** Exhaustive over all $3^4 = 81$ allocations (`routeA_cex.py`): 19 are good, and the minimum arc weight over them is $-3$, attained at $(\{g_1, g_3, g_4\}, \{g_2\}, \emptyset)$, with every other good allocation reaching $-2$. □

**Remark 68** (What this rules out)**.** Corollary 29 is a bound on *arcs*; the target is a bound on *paths*. Propositions 66 and 67 show the gap between the two is not slack that a cleverer construction can close — it is where all the surviving witnesses live. **Any correct invariant must be path-based.** That is precisely what makes agent induction hard, because path weights depend on which agent holds which bundle, so the invariant cannot be assignment-invariant and the newcomer's arrival re-opens the whole graph. Agent induction is not refuted here; the only invariant that would have made it easy is.

**Remark 69** (The obstruction is hypergraph discrepancy)**.** For binary additive costs, uniform balance says every $D_i$ is split evenly across the $n$ bundles, which is a discrepancy condition on the set system $\{D_1, \ldots, D_n\}$. Proposition 66 is a small discrepancy obstruction, and it is the same combinatorial phenomenon that drives the reduction by which [BSV21] proves deciding exact envy-freeness for binary additive chores NP-complete. The certified-hard set-splitting instance recorded from that reduction also admits no uniformly balanced family, checked by `routeA_adv.py`. So the balance signal of Remark 48, which three independent routes had pointed at, cannot be imposed exactly — only approximately, and the approximation is exactly the path slack.

### 10.3 The $n = 3$ specialisation

Since the attack was aimed at $n = 3$ first, it is worth recording exactly what the target reduces to there. The point is that with three agents the path structure is finite and small, so "$\ell \le 1$" becomes a fixed list of inequalities.

**Proposition 70** (Characterisation at $n = 3$)**.** *Let $n = 3$. An allocation $\mathcal{A}$ is good — envy-freeable with $\ell_{\mathcal{A}} \le 1$ — if and only if*

*(i) every directed cycle of $G_{\mathcal{A}}$ has weight at most 0;*

*(ii) every arc has weight at most 1;*

*(iii) every directed path of length two has weight at most 1.*

**Proof** Condition (i) is envy-freeability, by Theorem 10. Given it, no positive cycle exists, so $\ell_{\mathcal{A}}(i)$ is the maximum weight of a *simple* path leaving $i$; on three vertices such a path has length 0, 1 or 2, contributing weight 0, an arc, or a two-path respectively. So $\ell_{\mathcal{A}} \le 1$ is exactly (ii) together with (iii). □

**Corollary 71** (Only six inequalities, and most are free)**.** *Let $n = 3$ and let $\mathcal{A}$ satisfy (i) and (ii). For distinct $i, j, k$ the three-cycle bound gives $w_{\mathcal{A}}(i,j) + w_{\mathcal{A}}(j,k) \le -w_{\mathcal{A}}(k,i)$, so the two-path $i \to j \to k$ has weight at most 1 whenever $w_{\mathcal{A}}(k,i) \ge -1$. As $(i,j,k)$ runs over the six orderings of $N$, the closing arc $(k,i)$ runs over all six ordered pairs exactly once. Hence exactly those two-paths whose closing arc lies below $-1$ require separate checking, one per offending arc. In particular, if every arc lies in $[-1,1]$ then (iii) is automatic and $\mathcal{A}$ is good.*

Both statements are machine-checked against the true longest-path computation over $5.8 \times 10^5$ (instance, allocation) pairs — exhaustively at $n = m = 3$ and randomised to $m = 6$ — by `n3check.py`.

Corollary 71 is the sharpest handle available at $n = 3$, and it also explains precisely why Section 10 failed. The clause "if every arc lies in $[-1,1]$" is the uniform-balance hypothesis; the residual work is the two-paths whose closing arc is very negative; and Proposition 67 says that on the counterexample *every* good allocation has such an arc, so the residual work is not a corner case but the whole problem.

### 10.4 What survives: the two-tier restatement

One reformulation does survive, and it is worth recording because it is equivalent rather than merely sufficient.

**Proposition 72.** *Let $\mathcal{A}$ be envy-freeable with every arc at most 1. Then $\mathcal{A}$ is good if and only if the digraph of weight-1 arcs contains no directed path of length two.*

**Proof** If no such path exists then every directed path carries at most one positive arc, so has weight at most 1, and $\ell_{\mathcal{A}} \leq 1$. Conversely if $\ell_{\mathcal{A}} \leq 1$ then $p^* \in \{0,1\}^n$ by Observation 12; writing $S = \{i : p_i^* = 1\}$, Proposition 14 puts every arc within $S$, within $N \setminus S$, or from $N \setminus S$ into $S$ at weight at most 0, so weight-1 arcs run only from $S$ to $N \setminus S$ and cannot be composed. □

So the target is exactly: *find an envy-freeable allocation together with a bipartition of the agents such that all strict envy runs from one side to the other*. This is Proposition 14 again, now as a criterion rather than a certificate format. It is not assignment-invariant, consistent with Remark 68, but it is a condition on a bipartition rather than on a schedule, so it inherits none of the dead ends of Section 8 and none of the ex-ante failures of Section 9.

Two facts about the paid set are worth recording before that route is attempted. The first is a theorem and constrains what the set can be; the second is an observation from Section 12 and suggests where to look.

**Proposition 73** (Paid agents are those somebody finds expensive)**.** *Let $\mathcal{A}$ be envy-freeable. If $p_i^* \geq 1$ then there is an agent $v \neq i$ with $c_v(A_i) \geq c_v(A_v) + 1$.*

**Proof** Immediate from Theorem 28: $\ell_{\mathcal{A}}(i) \leq \max(0, \max_{v \neq i}[c_v(A_i) - c_v(A_v)])$, so $\ell_{\mathcal{A}}(i) \geq 1$ forces the inner maximum to be at least 1. □

So the paid set is contained in the set of agents whose bundle looks strictly more expensive to somebody than that somebody's own — which is exactly condition (c) of Proposition 14 read backwards. An agent cannot be paid merely for holding a bundle she herself dislikes; somebody else has to corroborate.

**Remark 74** (Where the paid set sits, empirically)**.** Across the 164 adversarial instances of Remark 90 that genuinely require a subsidy, the minimum-size paid set of the witness found is in every case contained in $\arg\max_i c_i(A_i)$ — the paid agents are among those carrying the largest own cost. That is 164 of 164, but it is a property of a witness selected by enumeration order rather than a proved rule, and own costs of different agents are measured by different functions, so it is a lead and not a theorem. Combined with Proposition 73, the natural rule to test first is: *pay exactly the agents whose bundle some other agent finds at least one unit more expensive than her own, and check whether the resulting bipartition admits an allocation satisfying the three two-tier conditions.*

## 11 Approach 6: A Pure Goods Reformulation, and a Working Algorithm

Every approach so far has worked inside the chore model or the replica model. This one leaves both, and it ends in a concrete algorithm with no observed failures: a construct-

and-repair procedure that has solved every one of 389,215 test instances tried against it, including every adversarial instance that defeated Approaches 1 through 5.

### 11.1 The transform is an involution

The size-shift transform of Theorem 15 is not merely a comparison device: it is an involution of the dichotomous class onto itself, so the chore problem can be restated as a question about *goods* with no chores, no replica, no coverage constraint and no schedule anywhere in it.

**Lemma 75.** *The map $c \mapsto \tilde{v}$, $\tilde{v}(S) := |S| - c(S)$, is an involution of the set of dichotomous set functions (Theorem 7) onto itself.*

**Proof** $\tilde{v}(\emptyset) = 0$, and for $g \notin S$ the marginal is $\tilde{v}(S \cup \{g\}) - \tilde{v}(S) = 1 - \big(c(S \cup \{g\}) - c(S)\big) \in \{0, 1\}$, so $\tilde{v}$ is dichotomous. Applying the map twice returns $|S| - (|S| - c(S)) = c(S)$. □

### 11.2 The target

Fix an allocation $\mathcal{A}$ and write $\tilde{p}$ for the minimum subsidy vector of the goods instance $\{\tilde{v}_i\}$ at $\mathcal{A}$, defined exactly when the chore instance is envy-freeable at $\mathcal{A}$, since by Theorem 15 the two envy graphs have the same cycle weights. Put $q_i := \tilde{p}_i + |A_i|$ — goods subsidy plus bundle size.

**Conjecture 76** (Target G)**.** *Every dichotomous goods instance admits an allocation whose vector $q$ has spread at most* 1*.*

**Proposition 77.** *Conjecture 76 implies Conjecture 2.*

**Proof** By Theorem 15, $\ell_{\mathcal{A}}(u) = \max_v [\tilde{d}_{\mathcal{A}}(u, v) + |A_u| - |A_v|]$, where $\tilde{d}_{\mathcal{A}}(u, v)$ is the heaviest $u \to v$ path in the goods envy graph. Appending to any $u \to v$ path a heaviest path leaving $v$ gives $\tilde{d}_{\mathcal{A}}(u, v) + \tilde{p}_v \le \tilde{p}_u$, so $\tilde{d}_{\mathcal{A}}(u, v) \le \tilde{p}_u - \tilde{p}_v$ and therefore $\ell_{\mathcal{A}}(u) \le q_u - \min_v q_v$. A spread of at most 1 therefore gives $\ell_{\mathcal{A}} \le 1$, and Observation 12 upgrades this to $p^* \in \{0, 1\}^n$ with total at most $n - 1$. □

Target G quantifies over *all* allocations, and $\tilde{p}$ is itself a longest-path weight, so Target G is a condition on paths rather than arcs — exactly what killed Approach 5 (Remark 68). It also survives extensive testing on its own: 0 failures on the full exhaustive $n = m = 3$ family and on the adversarial families that refuted the invariant of Approach 5, detailed in `targetG.py` and `targetG_adv.py`. But Target G alone does not say how to *find* the allocation. The rest of this section does.

### 11.3 Item-by-item insertion is the wrong template here too

The obvious next move is to run the algorithm of [BKNS22] directly on the size-shifted goods instance $\{\tilde{v}_i\}$, since it is a legitimate dichotomous instance and the algorithm's correctness proofs (Lemmas 3, 7, 8, 9, 10, 11 of that paper) are generic over *which* valid choice Extend and FindSink make at each step: any valid $(\rho, \kappa)$ pair, any starting agent in $\mathrm{M}(p)$. That freedom can be spent trying to steer bundle sizes toward balance while $\tilde{p} \in \{0, 1\}^n$ is preserved automatically.

It does not work. Greedily steering the choice that locally minimises $q$-spread fails on 1,635 of the 9,880 exhaustive $n = m = 3$ instances with a fixed item order, and on 440 of them even after trying every item order. Worse than a weak heuristic: on the smallest such instance, a full backtracking search over *every* legal execution of the algorithm — every item order, every Extend choice, every FindSink starting point, 474 states in total — never reaches spread $\leq 1$, although spread 0 is achievable (the perfectly balanced allocation $\{0\}, \{1\}, \{2\}$) and is confirmed reachable by direct, algorithm-free enumeration. This is a genuine reachability obstruction, verified by `guidedR3_full.py` and `verify_reach_gap.py`, in the same family as the peel dead ends of Section 8.5: item-by-item insertion fixes which items end up sharing a bundle the moment they are grouped, and no later permutation can split a bundle back apart, so an early bad grouping can be permanent.

The lesson is to stop building the allocation one item at a time. The rest of this section instead *constructs* a candidate directly and *repairs* it if needed — an operation insertion cannot perform, since repair means moving an item out of a bundle it is already in.

## 11.4 Construct and repair

**Definition 78** (Target G-bal)**.** *A partition of the items into $n$ groups is* cardinality-balanced *if the group sizes differ by at most* 1. Target G-bal *is the statement that every dichotomous goods instance admits a cardinality-balanced partition whose optimal (maximum-welfare) assignment of groups to agents gives $q$-spread at most* 1.

**Observation 79.** *Target G-bal implies Target G, since a cardinality-balanced partition is in particular a partition.*

Cardinality-balanced partitions are exactly what the balanced-matching machinery of [BDN+20, LMS26] is built to produce, so this restriction moves onto ground the literature already occupies. The algorithm below constructs one directly, using no chores, no replica, and no item-by-item insertion order.

**Algorithm (construct-and-repair).**

1. *Construct.* Run one pass of R11's iterated maximum-weight perfect matching (IMWPM): pad the items with inert dummies to a multiple of $n$, and in each of $\lceil m/n \rceil$ rounds assign every agent exactly one remaining item (real or dummy), by a matching maximising the total *marginal* value $\tilde{v}_i(A_i \cup \{g\}) - \tilde{v}_i(A_i)$ over the current partial bundles — the non-additive generalisation of the item-value weights of [BDN+20], needed because $\tilde{v}_i$ need not be additive. Discard the dummies.
2. *Repair.* While the current partition's optimal assignment has $q$-spread greater than 1: among all single-item swaps between two groups that keep the sizes cardinality-balanced, take the one that most reduces the ($q$-spread, $-$welfare) pair, re-optimise the assignment, and repeat.
3. *Restart on failure.* If repair reaches a local optimum with spread still above 1, restart step 1 from a freshly shuffled round-robin partition instead of IMWPM's output, and repeat step 2. Try up to a small constant number of restarts.

Every step preserves cardinality balance, so termination at spread $\leq 1$ directly witnesses Target G-bal — and hence, by Proposition 77 and Observation 79, produces an envy-free chore allocation with $p^* \in \{0,1\}^n$ once translated back through Theorem 15.

### 11.5 Evidence: zero failures across 389,215 instances

| Test set | Instances | Failures |
|---|---|---|
| Exhaustive, $n = m = 3$, all instances | 9,880 | 0 |
| Exhaustive, structured non-additive triples, $n = 3$, $m \in \{3,4,5\}$ | 378,176 | 0 |
| Every named hard instance in this project (Theorem 30's witness, Theorem 58's no-go instance, Proposition 66's discrepancy counterexample, Proposition 33's instance) | 4 | 0 |
| Randomised, $n \in \{3,\ldots,8\}$, $m \in \{3,\ldots,10\}$, endpoint-constant biased | 1,155 | 0 |
| **Total** | **389,215** | **0** |

Two components of this deserve to be separated out, since the construction step and the repair step have different track records on their own.

*IMWPM alone*, with no repair, already succeeds on all 9,880 exhaustive $n = m = 3$ instances and on all four hard instances, but fails on roughly 18% of larger randomised instances (132 of 720 tested) — it is greedy and short-sighted, so it can lock in a bad grouping in an early round exactly as insertion does, just less often.

*Repair from a bare round-robin start*, with no IMWPM warm start, reaches spread $\leq 1$ on all 9,880 exhaustive instances and on 357,746 of the 357,760 structured triples at $m = 5$; the remaining 14 are genuine local optima of the swap neighbourhood — confirmed by independent exhaustive search over all cardinality-balanced partitions, which finds spread 0 achievable on every one of them (`classify_failures.py`) — and every one of the 14 is recovered by restarting from a freshly shuffled partition, using at most 4 restarts (`restart_check.py`).

Combining both, as in Section 11.4, removed every failure found by either component alone, across every test run against it.

### 11.6 Complexity, and what is proved unconditionally

Three properties of the algorithm are theorems, not conjectures: its outputs are certified correct, it terminates in polynomial time, and its search space is genuinely the set of *unordered* balanced partitions. We state them with proofs, in the value-oracle model with per-query cost $\tau$.

**Lemma 80** (Assignment invariance)**.** *Fix an unordered family of $n$ bundles. The assignments of bundles to agents under which the allocation is envy-freeable are exactly the welfare-maximising ones, and every welfare-maximising assignment induces the same minimum subsidy on each*

bundle *[KMS+24, Lemma 2]. Hence the multiset* $\{\tilde{p}(B) + |B|\}$*, and with it the q-spread, depends only on the unordered partition.*

**Proof** The first claim is condition (ii) of Theorem 10 applied to the goods instance: an assignment is envy-freeable iff no reassignment of its own bundles raises welfare, which is exactly welfare maximality over assignments. The second is Lemma 2 of [KMS+24], which proves that the minimum subsidy vectors of any two maximum-weight assignments coincide bundle-by-bundle. Adding $|B|$, a bundle attribute, preserves this. □

Two consequences. First, the $n!$ assignment loop in the implementation is mathematically equivalent to a single maximum-weight matching computation, so the complexity below is stated with the Hungarian algorithm, $O(n^3)$. Second, the algorithm's true search space is unordered balanced partitions — the object Conjecture 88 should be read as quantifying over.

**Theorem 81** (Soundness, certified)**.** *Suppose the algorithm outputs a partition and assignment with q-spread at most* 1*. Then the* same *allocation, read in the original chore instance — the size-shift acts on valuations only, so the allocation needs no translation — together with* $p := \ell_{\mathcal{A}}$ *is an envy-free solution with* $p \in \{0,1\}^n$ *and total subsidy at most* $n - 1$*. Moreover the output is verifiable in* $O(n^2\tau + n^3)$ *time independently of how it was found.*

**Proof** Proposition 77 gives $\ell_{\mathcal{A}} \le 1$ on the chore side; Observation 12 upgrades to $p^* \in \{0,1\}^n$; the endpoint of a heaviest path has $\ell = 0$, giving total at most $n - 1$. Verification recomputes the $n^2$ bundle values ($n^2$ oracle calls), one maximum-weight matching and one longest-path computation ($O(n^3)$ each). □

**Theorem 82** (Termination and complexity)**.** *On any dichotomous instance:*

*(i) evaluating one partition costs* $O(n^2\tau + n^3)$*;*

*(ii) the construction pass (IMWPM) costs* $O(m^2\tau + nm^2)$*;*

*(iii) each repair iteration examines* $O(nm)$ *moves and costs* $O(n^3m\tau + n^4m)$*;*

*(iv) the repair loop performs at most* $O(m^2)$ *iterations, so with a constant number of restarts the whole algorithm runs in time* $O(n^3m^3\tau + n^4m^3)$*.*

*In particular the algorithm always terminates, in polynomial time, regardless of whether it succeeds.*

**Proof** (i) is the count in Theorem 81, using Lemma 80 to replace the assignment loop by one Hungarian computation. (ii): there are $\lceil m/n \rceil$ rounds; each builds an $n \times O(m)$ marginal table ($O(nm)$ oracle calls, two per entry) and solves one rectangular assignment problem in $O(n^2m)$; summing gives the bound. (iii): a move sends one of $m$ items to one of $n - 1$ other groups, and each candidate is evaluated by (i). (iv) is the substantive claim. Because marginals lie in $\{0,1\}$, every bundle value satisfies $\tilde{v}_i(B) \le |B|$; hence total welfare is an integer in $[0, m]$, and every subsidy $\tilde{p}_i$, being a path weight, is an integer in $[0, m]$, so the spread is an integer in $[0, m+1]$. The repair loop moves only when the pair (spread, −welfare) strictly decreases lexicographically, and that pair takes at most $(m+2)(m+1)$ values, so at most $O(m^2)$ moves occur before the loop halts — either at spread $\le 1$ or at a local optimum, where a restart or a failure report follows. □

The gap between these theorems and a full correctness proof is exactly Conjecture 88: soundness says the algorithm never lies, termination says it always answers, and what is missing is that the answer is always "success." For two agents, that missing piece can be supplied.

### 11.7 A complete proof for two agents

**Theorem 83** (Completeness at $n = 2$)**.** *Every two-agent dichotomous goods instance admits a cardinality-balanced split with $q$-spread at most 1. Consequently Target G-bal, Target G, and Conjecture 2 all hold at $n = 2$ — the last with the additional guarantee, not provided by the complementation route of Remark 9, that the two bundles differ in size by at most one.*

Fix dichotomous $\tilde{v}_1, \tilde{v}_2$ on $M$, $|M| = m$. For an *ordered* balanced split $(A, B)$ with $|A| = \lceil m/2 \rceil \geq |B|$, write

$$\alpha := \tilde{v}_1(A) - \tilde{v}_1(B), \qquad \beta := \tilde{v}_2(A) - \tilde{v}_2(B), \qquad u := \tilde{v}_1 + \tilde{v}_2, \qquad h := u(A) - u(B) = \alpha + \beta.$$

Call the split *good* if some envy-freeable orientation (assignment of $A, B$ to the two agents) has $q$-spread at most 1.

**Lemma 84** (Swap step)**.** *Exchanging one item of $A$ with one item of $B$ changes each of $\tilde{v}_i(A)$ and $\tilde{v}_i(B)$ by at most 1, hence each of $\alpha, \beta$ by at most 2 and $h$ by at most 4.*

**Proof** Removing an item changes a dichotomous value by 0 or $-1$ and adding one by 0 or $+1$, so the composite changes each side's value by at most 1 in absolute value. □

**Lemma 85** (Window)**.** *If $m$ is even and $|h| \leq 3$, or $m$ is odd and $-1 \leq h \leq 3$, then the split is good.*

**Proof** *Even $m$.* Sizes are equal, so $q$-spread equals $\left|\tilde{p}_1 - \tilde{p}_2\right|$, and it suffices to find an envy-freeable orientation in which both envies are at most 1. Giving $A$ to agent 1 yields envies $(-\alpha, \beta)$, so that orientation is satisfactory iff $\alpha \geq -1$ and $\beta \leq 1$; the flipped orientation iff $\alpha \leq 1$ and $\beta \geq -1$. Both fail only when $\alpha, \beta \leq -2$ or $\alpha, \beta \geq 2$, and in either case $|h| \geq 4$. It remains to check envy-freeability. The two-cycle of the first orientation has weight $\beta - \alpha$; if it is positive while that orientation is satisfactory ($\alpha \geq -1$, $\beta \leq 1$, $\beta > \alpha$), then $\alpha < \beta \leq 1$ and $\beta > \alpha \geq -1$ show the flipped orientation is satisfactory as well, and its cycle $\alpha - \beta$ is negative. So a satisfactory envy-freeable orientation exists, its subsidies lie in $\{0, 1\}$, and the spread is at most 1.

*Odd $m$.* Now $|A| = |B| + 1$, and with subsidies in $\{0, 1\}$ the spread is at most 1 iff the holder of the large bundle needs no subsidy — both agents needing 1 is impossible, since then the two-cycle $\geq 2$ would be positive. Giving $A$ to agent 1 is satisfactory iff $\alpha \geq 0$ and $\beta \leq 1$; flipped, iff $\beta \geq 0$ and $\alpha \leq 1$. Both fail only when $\alpha, \beta \leq -1$ or $\alpha, \beta \geq 2$, i.e. when $h \leq -2$ or $h \geq 4$. Envy-freeability is repaired exactly as in the even case: if the first satisfactory orientation has $\beta > \alpha$, then $\alpha < \beta \leq 1$ gives $\alpha \leq 0$ and $\beta > \alpha \geq 0$ gives $\beta \geq 0$, so the flipped orientation is satisfactory with a negative cycle. □

**Proof of Theorem 83** *Even $m$.* Take any equal split, ordered so that $h \geq 0$. If $h \leq 3$, Lemma 85 finishes. Otherwise $h \geq 4$; walk to the complementary split $(B, A)$ by exchanging one paired item at a time ($m/2$ swaps). The final value is $-h \leq -4$, and by Lemma 84 each

step changes $h$ by at most 4, so at the first step where the value drops below 4 it lies in $[0, 3]$ — inside the window.

*Odd $m$.* The big sides are the $(k{+}1)$-subsets, $m = 2k + 1$. Suppose no split is good; then by Lemma 85 every big side $A$ has $h(A) \le -2$ or $h(A) \ge 4$. Any two big sides are connected by single exchanges (repeatedly swap an element of the difference), and a step from $h \ge 4$ to $h \le -2$ would change $h$ by at least $6 > 4$, so *all* big sides lie on one side. If all have $h \ge 4$: let $A^*$ minimise $u$ over big sides, and pick any $x \in A^*$. Then $B := (M \setminus A^*) \cup \{x\}$ is a big side with $u(B) \le u(M \setminus A^*) + 2 \le u(A^*) - 2$, contradicting minimality — the first inequality because $u$ has marginals in $\{0, 1, 2\}$, the second because $h(A^*) \ge 4$. If all have $h \le -2$: let $A^\circ$ maximise $u$ over big sides and pick any $x \in A^\circ$. Then $u((M \setminus A^\circ) \cup \{x\}) \ge u(M \setminus A^\circ) \ge u(A^\circ) + 2$, by monotonicity and $h(A^\circ) \le -2$, contradicting maximality. □

**Corollary 86** (An exact $O(m\tau)$ algorithm for two agents)**.** *For $n = 2$ the proof is constructive and needs no restarts: start from any balanced split; if it is not good, then for even $m$ walk toward the complement, and for odd $m$ first apply the jump $A \mapsto (M \setminus A) \cup \{x\}$ — which maps $h \ge 4$ to $h \le 0$ and $h \le -2$ to $h \ge 2$ — and then, if the jump target is still outside the window, walk between the two splits. Some split within $m/2 + 2$ evaluations lies in the window and is good. Each evaluation uses $O(1)$ oracle calls, so the whole procedure runs in $O(m\tau)$ time.*

**Proof** The even case is the walk in the proof above. For the odd jump, write $B = (M \setminus A) \cup \{x\}$ with $x \in A$, so $M \setminus B = A \setminus \{x\}$ and $h(B) = u(B) - u(A \setminus \{x\})$. Recall that $u = \tilde{v}_1 + \tilde{v}_2$ has marginals in $\{0, 1, 2\}$, so adding or removing a single item changes $u$ by at most 2. If $h(A) \ge 4$ then $u(B) \le u(M \setminus A) + 2 \le u(A) - 2$ while $u(A \setminus \{x\}) \ge u(A) - 2$, giving $h(B) \le 0$. If $h(A) \le -2$ then $u(B) \ge u(M \setminus A) \ge u(A) + 2$ while $u(A \setminus \{x\}) \le u(A)$, giving $h(B) \ge 2$. Both bounds are attained, so neither can be sharpened. In either case the jump target is in the window $[-1, 3]$ or strictly on the far side of it, and in the latter case the swap walk between $A$ and $B$ crosses the window as in the theorem: descending from $\ge 4$, the first value below 4 is at least 0; ascending from $\le -2$, the first value above $-2$ is at most 2. □

The theorem and the walk are machine-verified by `n2proof_check.py`: exhaustively over all instances at $m \le 4$ (490,545 pairs at $m = 4$ alone) and on 30,000 random instances at $m \in \{5, 6, 7\}$ — 0 failures of either, with the walk never needing more than 2 evaluations in practice against its $m/2 + 2$ bound.

**Remark 87** (What the proof teaches the algorithm)**.** Two findings from the proof effort feed back into the implementation. First, the moves that the $n = 2$ proof walks are *pairwise exchanges*, which preserve group sizes exactly — but the implemented repair neighbourhood contains only single-item *moves*, which change two group sizes by one. When $n$ divides $m$, every group has size $m/n$ and every single move is rejected by the balance filter, so the repair loop is inert there and the implementation succeeds purely through IMWPM plus restarts; the zero failures at $n = m$ and at $(n, m) \in \{(3, 6), (6, 6), (7, 7), (8, 8)\}$ were all achieved in that degenerate mode. Adding exchanges to the neighbourhood is therefore both principled — they are the proof's own moves — and necessary for the repair step to act at all when $n \mid m$. Second, the $h$-window technique suggests the right potential for a general proof: a one-dimensional aggregate whose swap-Lipschitz constant is smaller than the width of the region it must cross. At $n = 2$ the aggregate $u = \tilde{v}_1 + \tilde{v}_2$ works because

the goodness condition is two-sided in a single difference; for $n \geq 3$ goodness involves $\binom{n}{2}$ pairwise differences, and no single aggregate is yet known to control them.

### 11.8 What remains

After Sections 11.6 and 11.7, the standing of the approach is: soundness, certification, termination and polynomial complexity are theorems for every $n$; completeness is a theorem for $n = 2$; and for $n \geq 3$ completeness is the one remaining open statement, sharp and almost entirely combinatorial, with no chores, no replica and no scheduling left in it:

**Conjecture 88** (Construct-and-repair succeeds, $n \geq 3$)**.** *For every dichotomous goods instance, the algorithm of Section 11.4 — with pairwise exchanges added to the repair neighbourhood, per Remark 87 — reaches a cardinality-balanced partition with q-spread at most* 1*, using a number of restarts bounded by a constant independent of n and m.*

The 14 local optima found at $n = 3$ show that the move neighbourhood does contain bad local optima, so a proof cannot simply be "every non-optimal balanced partition has an improving move." What was observed instead is that bad local optima are escapable from other starting partitions, always within a handful of restarts. The $n = 2$ proof suggests what to look for: a one-dimensional aggregate — there, $u = \tilde{v}_1 + \tilde{v}_2$ with the window $|h| \leq 3$ — whose per-move variation is smaller than the forbidden gap it must cross, together with an extremal argument at the ends. Finding the analogous aggregate for $n \geq 3$, where goodness involves $\binom{n}{2}$ pairwise comparisons rather than one, is the whole remaining problem; solving it would upgrade Conjecture 88, and with it Conjecture 2, from extensively tested to proved.

## 12 Computational Evidence

All searches operate directly on the graph-theoretic form of Conjecture 2: enumerate allocations, build the envy graph, detect positive-weight cycles and compute $\ell_{\mathcal{A}}$ by a Bellman–Ford iteration on maximum-weight walks, and record $\min_{\mathcal{A}} \max_i \ell_{\mathcal{A}}(i)$.

| Experiment | Script | Result |
|---|---|---|
| Exhaustive: $n = m = 3$, all 9,880 instances up to agent symmetry | `search.py` | worst-case minimum per-agent subsidy = 1 |
| Randomised: $n \in \{3,4,5\}$, $m \in \{4,5,6\}$, $\approx 3.5 \times 10^4$ instances | `fast.py` | no counterexample |
| Binary additive construction of Theorem 18, $2 \times 10^5$ instances, $n \le 5$, $m \le 7$ | `binadd.py` | maximum per-agent subsidy ever observed = 1 |
| The obstruction of Theorem 30 | `deadend.py` | all three insertions require subsidy 2; full instance solvable with 0 |
| Does *some* utilitarian-optimal allocation attain $\le 1$? | `msw.py` | yes, in all but one of $\approx 2.6 \times 10^4$ instances |
| Does *every* utilitarian-optimal allocation attain $\le 1$? | `msw.py` | no — fails in $\approx 2.5\%$ of instances at $n = 3$, $m = 4$, rising to $\approx 5\%$ at $m = 5$ |
| The single exception | `mswcex.py` | refutes Conjecture 17 (Proposition 33) |
| Peel schedule and trap-state refusal on the witness of Theorem 30 | `peel.py` | six legal peels reach the zero-subsidy allocation; the trap state has $\ell(1) = 2$ |
| Full peel state space of that witness, $7^3 = 343$ states, both move types | `peel3.py` | 175 legal, 166 good, **9 dead ends**, root good, all six reachable terminals are perfect matchings |

and, for the approaches attempted after the replica transform,

| Experiment | Script | Result |
|---|---|---|
| No dichotomous reweighting separates: all $990^3$ triples on the instance of Theorem 58, plus the separable $P$-family on three instances | `dupsep.py` | **0** separating triples (Theorem 59) |
| Type-ordered peel schedules, witness plus 890 random instances, $n \in \{3,4\}$, $m \le 4$ | `typeorder.py`, `stress.py` | every instance admitting a legal schedule admits a type-ordered one; arg max recipient rule fails (Proposition 52) |
| Double-refinement selection rule, $\approx 1050$ random instances | `rules.py` | no failure — but see the next row |
| The same rule on the instance of Proposition 33 | `checkD.py` | **fails**: selected allocation needs 2, another needs 0 (Remark 36) |
| Iterated minimum-marginal perfect matching, non-additive costs | `rules.py` | fails on 13/250 at $n = 3, m = 4$ and 10/120 at $n = 3, m = 5$ |
| Uniformly balanced families: exhaustive $n = m = 3$, then $\approx 1900$ random instances up to $n = 5$ | `routeA.py` | no failure, and the implication chain verified on every instance |
| The same, hunted adversarially over structured families | `routeA_adv.py` | 10 **failures** at $n = 3, m = 4$, plus the certified-hard set-splitting instance (Proposition 66) |
| Arc weights of the good allocations on that counterexample | `routeA_cex.py` | all 19 have an arc $\le -2$; minimum $-3$ (Proposition 67) |
| **Conjecture 2 itself**, hunted adversarially over non-additive instances: exhaustive structured triples at $m = 3, 4$ plus $\approx 8.4 \times 10^3$ endpoint-constant instances to $n = 5$ | `twotier.py` | **no counterexample** (Remark 89) |
| How often a subsidy is needed at all | `twotier.py` | exactly envy-free in 98.6% of 11,884 instances (Remark 90) |
| Structure of the paid set on the 164 instances that need one | `hardcases.py` | min $\lvert S\rvert \le 3$, always inside $\arg\max_i c_i(A_i)$ (Remark 74) |
| The $n = 3$ characterisation, against the true longest-path computation | `n3check.py` | 0 mismatches over $5.8 \times 10^5$ (instance, allocation) pairs (Proposition 70) |

and, for the construct-and-repair algorithm of Section 11,

| Experiment | Script | Result |
|---|---|---|
| Item-by-item insertion on the transformed goods instance, greedily steered | `guidedR3.py`, `guidedR3_test.py` | 1,635/9,880 (fixed order), 440/9,880 (best of 6) |
| Full backtracking over every legal insertion execution on the smallest failure | `guidedR3_full.py` | spread 2 is the best reachable; spread 0 is unreachable |
| Independent, algorithm-free confirmation that spread 0 is achievable there | `verify_reach_gap.py` | confirmed — a genuine reachability obstruction |
| IMWPM warm start alone, no repair | `imwpm_raw.py` | exhaustive $n = m = 3$ and all hard instances pass; $\approx 18\%$ of larger random instances fail |
| Swap-based repair from a round-robin start alone | `targetGbal_local.py` | 9,880/9,880 exhaustive; 357,746/357,760 structured triples at $m = 5$ |
| The 14 residual local optima, classified | `classify_failures.py` | all 14 are local-search artefacts — exhaustive balanced search finds spread 0 on every one |
| The same 14, retried with shuffled restarts | `restart_check.py` | 14/14 recovered, at most 4 restarts each |
| **The combined algorithm**: IMWPM warm start, swap repair, restart on failure | `final_algorithm.py` | 0 **failures across** 389,215 **instances** — exhaustive $n = m = 3$, exhaustive structured triples to $m = 5$, every named hard instance, and randomised sweeps to $n = 8, m = 10$ |
| The $n = 2$ completeness theorem and its $O(m\tau)$ constructive walk | `n2proof_check.py` | 0 failures over 521,310 instances — exhaustive $m \le 4$ (490,545 pairs at $m = 4$), random $m \in \{5, 6, 7\}$; walk never exceeded 2 evaluations (Theorem 83, Corollary 86) |

The exhaustive row is the only unconditional evidence for Conjecture 2. The dichotomous functions on $m$ items number $2, 6, 38, 990$ for $m = 1, 2, 3, 4$. Agents are interchangeable, so an instance is a multiset of three of them, and at $m = 3$ there are $\binom{38+3-1}{3} = \binom{40}{3} = 9{,}880$ of these — feasible to enumerate, and already out of reach at $m = 4$.

The last three rows are the ones that changed the shape of the problem. The utilitarian-optimal set almost always contains a witness, but not always, and one exception is enough: Proposition 33 is built from the single instance in which it did not.

### 12.1 Why the randomised evidence is weak

The count of $\approx 3.5 \times 10^4$ instances without a counterexample overstates the support for Conjecture 2, and the reason is structural rather than a matter of sample size.

The generator builds a dichotomous function by visiting subsets in order of increasing size and, at each subset $S$, computing the interval $[\ell, h]$ of values consistent with what has already been fixed — namely $\ell = \max_{g \in S} c(S \setminus \{g\})$ and $h = \min_{g \in S} c(S \setminus \{g\}) + 1$ — then choosing an endpoint by an unbiased coin whenever $\ell \neq h$. Independent fair coins put almost all of the mass on functions that wander in the middle of the feasible region. Structured extremal functions, which take the same endpoint at every subset, are exponentially unlikely: on $m$ items there are $2^m - 1$ nonempty subsets and a function that requires a prescribed endpoint at each of them is sampled with probability $2^{-(2^m-1)}$, which is $1/128$ at $m = 3$ and about $3 \times 10^{-5}$ at $m = 4$, per agent.

This matters because the two instances that carry the results of this paper are of exactly that kind. The obstruction of Theorem 30 uses $c_1(S) = \max(0, |S| - 1)$ alongside $c_2 = c_3 = |S|$, all three of which are endpoint-constant; drawing that triple at $m = 3$ has probability $2^{-21} \approx 5 \times 10^{-7}$, so a run of $3.5 \times 10^4$ samples would be expected to produce it about once in every sixty runs. Both instances were found by hand or by a targeted search, not by the sampler. The sampler has, in effect, never looked in the region where the interesting behaviour lives.

**Remark 89** (The experiment worth running)**.** If Conjecture 2 is false, the counterexample is in the corner the current generator cannot reach, and no amount of additional uniform sampling will find it. An adversarial generator — biased towards endpoint-constant functions, towards supermodular costs, towards agents whose free chores overlap heavily, and towards instances in which the utilitarian optimum is unique — is the single highest-value experiment outstanding against the *conjecture*.

It has now been built (`routeA_adv.py`, `twotier.py`) and it works: it refuted the invariant of Section 10 in a single run, on an instance that $\approx 1.2 \times 10^4$ uniformly sampled instances had missed. Pointed at Conjecture 2 itself it finds nothing. Since binary additive instances are settled by [LMS26], a counterexample must be non-additive and the search was restricted accordingly: exhaustive over all structured non-additive triples at $m = 3$ (120 instances) and $m = 4$ (5,984), then $\approx 8.4 \times 10^3$ endpoint-constant instances up to $n = 5$, $m = 6$. **No instance without a good allocation was found.** This is the first evidence for the conjecture produced by a method demonstrated to find counterexamples on this problem, and it is a genuine update: the accumulating failures of Sections 6, 7, 9 and 10 are evidence about the *methods*, not about the statement.

**Remark 90** (Where the difficulty is concentrated)**.** The same sweep records how often a subsidy is needed at all. Across 11,884 adversarial non-additive instances an *exactly* envy-free allocation — zero subsidy — exists in 98.6% of them; only 164 require any subsidy, and among those the minimum number of paid agents is 1 or 2 at $n = 3$ and never exceeds 3 at $n = 4$.

Two cautions. The pool is adversarially structured rather than representative, so the percentage describes that pool and is not a claim about the model. And the residual 1.4% is not negligible in the way the number suggests: deciding whether an exactly envy-

free allocation exists is NP-complete already for binary additive chores [BSV21], so those instances are hard for a reason and they are precisely the ones any construction must handle.

### 12.2 The peel-frame search

The highest-value experiment against the *approach* of Section 8 is different, and it is cheap. Reuse the `gen.py/search.py` harness over the full $n = 3$, $m = 3$ dichotomous family — all 9,880 instances up to agent symmetry — and for each instance run the fixed-point computation of `peel3.py`: is the root good, and how many dead ends are there? An instance with a bad root refutes Conjecture 49 and kills the peel approach.

It would *not*, by Remark 50, refute Conjecture 2: a good terminal state can in principle exist while every schedule reaching it passes through an illegal intermediate state. The two searches therefore test different things and both are worth running — the allocation-space search above tests the conjecture, and the peel-frame search tests whether this route to it survives.

Two follow-ups, in order. First, test the hypothesis of Remark 48 across that family: is every dead end a state already committed to an unbalanced terminal? If so, formulate the balance rule precisely and check whether greedy-under-that-rule reaches a terminal without backtracking. Second, try candidate rules in the order (a) balance-first, peel so that a balanced terminal remains reachable; (b) spectator-free peels first by Lemma 44, ties broken by balance; (c) $\arg\max_i \ell(i)$ with a balance tie-break — noting that (c) alone already fails on the witness.

### 12.3 Reproducing the results

| Script | Purpose |
|---|---|
| `gen.py` | enumerate all dichotomous functions on $m$ items |
| `search.py` | exhaustive search, arguments `m n` |
| `fast.py` | randomised counterexample hunt, `m n T seed` |
| `msw.py` | utilitarian-optimal test, `m n T seed` |
| `tiebreak.py` | selection rules of Section 7.2 |
| `localsearch.py` | local minima of single-transfer search |
| `deadend.py` | Theorem 30 |
| `binadd.py` | Theorem 18 |
| `mswcex.py` | Proposition 33 |
| `peel.py` | the schedule and trap-state refusal of Section 8.4 |
| `peel3.py` | the state-space table of Proposition 47 |
| `dupsep.py` | Theorems 56, 58, 59; `dupsep.py pfamily` for the $P$-family sweeps |
| `typeorder.py` | type-ordered schedules on the witness and small families |
| `stress.py` | randomised type-ordering sweep, seeded |
| `rules.py` | the refinement rules and the matching lift |
| `checkD.py` | Remark 36 |

and, continuing,

| Script | Purpose |
|---|---|
| `routeA.py` | Proposition 65; exhaustive $n = m = 3$ and randomised sweeps |
| `routeA_adv.py` | adversarial hunt; finds Proposition 66 |
| `routeA_cex.py` | Propositions 66 and 67 |
| `twotier.py` | Remarks 89 and 90 |
| `hardcases.py` | Remark 74 |
| `n3check.py` | Proposition 70 and Corollary 71 |
| `monotone_check.py` | Remark 39: monotonicity alone does not suffice |
| `targetG.py` | Conjecture 76, exhaustive $n = m = 3$ |
| `targetG_adv.py` | Target G against the Approach 5 adversary |
| `guidedR3.py`, `guidedR3_test.py` | guided-insertion greedy; Section 11.3 |
| `guidedR3_full.py` | the insertion reachability obstruction |
| `verify_reach_gap.py` | algorithm-free confirmation of that gap |
| `targetGbal.py` | Definition 78, exhaustive over balanced partitions |
| `targetGbal_local.py` | the swap-based repair, round-robin start |
| `imwpm_raw.py` | the IMWPM warm start alone |
| `targetGbal_stress.py` | the stress sweep that found the 14 local optima |
| `classify_failures.py` | classifies the 14 as search artefacts |
| `restart_check.py` | confirms all 14 recovered by restarts |
| `final_algorithm.py` | the combined algorithm; the 389,215-instance sweep |
| `n2proof_check.py` | Theorem 83 and Corollary 86, exhaustive $m \leq 4$ plus random $m \leq 7$ |
| `balanced_msw.py`, `tiebreak_balanced.py`, `ruleD_adv.py` | the min-cost-balanced rule and its tie-breaks |
| `structural.py` | Theorem 22, Corollary 23, two-type sweep |

The instance of Proposition 33 was produced by `msw.py 4 3 3000 11` at trial 2460 and is re-derived independently by `mswcex.py`, which re-checks that all three cost functions are negative dichotomous, that the utilitarian optimum is unique, and that a zero-subsidy allocation exists outside it.

## 13 Ideas and Open Threads

A running log, newest last. Nothing in this section is claimed to be true.

### 2026-08-01 — What the two negative results jointly require

Taking Theorem 30 and Proposition 33 together, any correct algorithm must be free to (i) move already-allocated chores between bundles and (ii) give up total cost while doing so. Every construction that currently works does so by exhibiting a per-agent potential $\phi$ with $w_{\mathcal{A}}(i, j) \leq \phi_i - \phi_j$, which telescopes along paths and vanishes around cycles. *Finding*

*such a potential for general negative dichotomous costs, or proving that none exists, is the problem.* In Theorem 18 the potential is $\phi_i = u_i$, the count of universally disliked chores held; in Theorem 20 it is $\phi_i = c(A_i)$. What is the right $\phi$ in general?

Worth noting that the existence of such a $\phi$ is not merely sufficient but close to necessary: if $p^* \in \{0,1\}^n$ then $\phi_i := p_i^*$ satisfies $w_{\mathcal{A}}(i,j) \leq \phi_i - \phi_j$ by definition of an envy-free solution. So the search for a potential and the search for a proof are the same search, and the content is in constructing $\phi$ and the allocation *together*.

### 2026-08-03 — routes surviving the two no-go results

Section 9 closes valuation-level encoding and algorithm-level weight forcing, and Corollary 35 closes every rule that selects inside the utilitarian-optimal set. What remains splits by which of those failures it avoids.

**Route M — iterated matching.** [LMS26, §3] develops the objective-chores analogue of iterated maximum-weight *perfect* matching, padding with zero-valued dummies so every agent takes one object per round, which makes the output balanced. The obvious lift to our setting — each round assign every agent one remaining chore minimising the total marginal $\sum_i [c_i(A_i \cup \{g_i\}) - c_i(A_i)]$ — does *not* give $\ell \leq 1$ for non-additive dichotomous costs: it fails on 13/250 instances at $n = 3, m = 4$ and 10/120 at $n = 3, m = 5$ (`rules.py`). That is diagnostic rather than discouraging, and [LMS26] flags the obstacle itself: the telescoping needs additivity, because an additive round-$t$ cost is independent of what came before whereas a dichotomous marginal depends on the bundle already held. The repair to attempt is a round invariant replacing the additive domination — each round's matching minimum-cost *conditioned on the current bundles*, with marginals non-increasing across rounds, a "peel the free chores first" discipline. **Status:** the naive lift is refuted; the conditioned version is untested.

**Route A — induct on agents, not items.** Every approach so far inducts on items. [LMS26, §4.3.1] instead maintains a set of active agents, adds one at a time, and restores two invariants: compensated utility at least $\lambda$ with $\lambda \leq p_i \leq 1$, and an equality path from every active agent to the set of minimum compensated utility. For chores this is attractive because a newcomer starts with an empty bundle, which is the *cheapest* bundle, so the newcomer is the natural sink and the orientation problem of Section 6 does not arise. The invariant carried is the equality graph rather than a subsidy scalar — which is precisely the extra information Theorem 61 and Proposition 52 say a recipient rule needs. **Status:** untried, and the most promising of these.

**Route N — non-redundancy instead of coverage.** Theorem 61 says the failing placements are exactly the zero-marginal ones, and the literature already forbids those under the name non-wastefulness. A duplicate is redundant for its holder by construction, so ask for a *non-redundant* allocation of the replica set — which General Yankee Swap is built to maintain — and measure how far non-redundancy falls short of coverage. The two are not equivalent: a duplicate can be redundant for everyone when every other agent already holds the type or has zero marginal for it. But the residual gap is narrower than coverage itself and is exactly characterisable. **Status:** cheap to test, untested.

**Route X — matroid exchange.** As in Section 8.8. Highest ceiling, highest cost; the parallel-extension step needs a full proof before anything is built on it.

**Ranking.** Route A, then Route M's conditioned repair, then Route N, then Route X. Conjecture 51 — the type-ordered peel — stays live alongside them and has the best evidence of anything currently open.

### 2026-08-06 — a cleaner rule than construct-and-repair

**The idea.** *Among cardinality-balanced allocations of minimum total cost, take one minimising the number of envious ordered pairs* $\#\{(i,j) : c_i(A_i) > c_i(A_j)\}$. A single two-level optimisation: no local search, no restarts, no schedule, no warm start.

**Why it might work.** It sidesteps Corollary 35 because the counterexample there has a *unique* optimum of sizes $(0,3,1)$, which is not balanced — restricting the optimisation to balanced allocations removes it from the feasible set entirely. A cost-minimising balanced allocation is automatically envy-freeable, since any reassignment of its own bundles is another balanced allocation and so cannot be cheaper, which means only $\ell \le 1$ remains to be shown. The second criterion is the one aligned with Proposition 14: in a $\{0,1\}$ solution envy runs only from the paid tier to the unpaid one, so minimising envy count pushes towards exactly the two-tier shape.

**Status.** Zero failures in the *strong* form — every allocation the rule can select is good — over 41,196 instances: exhaustive over all dichotomous triples at $n = m = 3$, exhaustive over structured families at $m \le 4$, and endpoint-constant sweeps to $n = 5$, $m = 6$ (`update_8/ruleD_adv.py`). Without the tie-break the cost-minimising balanced set always *contains* a good allocation but is not uniformly good; the three other natural tie-breaks — leximin own-cost, minimum total perceived load, leximin perceived load — each fail at $n = 3$, $m = 4$ (`tiebreak_balanced.py`).

**What would kill it.** A balanced instance where every minimum-cost, minimum-envy allocation needs subsidy 2. **Not proved**, at any $n$: an attempt at $n = 3$ stalled on the exchange step, because dichotomous costs need not be submodular and a bundle of positive cost need not contain any single item whose removal lowers it — so "$c_u(A_u)$ is large, therefore some swap improves" is unavailable. That is the same missing exchange property that makes the matroid-rank subclass (Section 8.8) the natural next target: there it *is* available.

### 2026-08-06 — at most two distinct cost functions

**The idea.** Suppose the $n$ agents carry only *two* distinct cost functions $c$ and $c'$, in groups of sizes $k$ and $n-k$. This strictly generalises the identical-cost case of Theorem 20 and is not covered by anything cited here: the literature settles identical, additive and submodular costs, but "two types of agents" is a different slice — one that does appear in the house-allocation work of [DCW+24] as a tractable restriction, which is mild evidence it is the right kind of hypothesis.

**Why it might work.** The two-tier conditions collapse dramatically. For two agents of the same type in the same tier, (a) forces $c(A_i) = c(A_j)$ exactly; for two of the same type in different tiers, (b) and (c) force $c(A_i) = c(A_j) + 1$ exactly. So within each type group the bundle costs take at most two values, differing by exactly one, and the tier split inside that group is determined by which value a bundle takes. What remains is a finite system of

cross-group inequalities in two functions rather than $n$, which is the kind of thing a direct construction can attack.

**Status.** Untried as a proof. Zero failures over 7,805 two-type instances at $m \in \{3,4\}$, $n \in \{3,4\}$, both for existence and for the rule of the previous entry (`update_9/structural.py`). Worth one session: it is the first class in a while where the conditions genuinely simplify rather than merely being restricted.

**What would kill it.** A two-type instance with no good allocation. Note this would also refute Conjecture 2 outright, so the search is worth running for its own sake.

### 2026-08-06 — induct on the depth of the cost functions

**The idea.** Stop inducting on items, on agents, or on schedules, and induct on the *cost functions themselves*. Every dichotomous $c$ decomposes into nested monotone Boolean layers: putting $\mathcal{F}_t := \{S : c(S) \geq t\}$, each $\mathcal{F}_t$ is upward-closed, $\mathcal{F}_1 \supseteq \cdots \supseteq \mathcal{F}_K$ with $K = c(M)$, and

$$c(S) \;=\; \#\{t : S \in \mathcal{F}_t\} \;=\; \sum_{t=1}^{K} \mathbb{1}\big[S \in \mathcal{F}_t\big].$$

Crucially the class is closed under truncation: $\min(c, k)$ is dichotomous for every $k$, so layers can be peeled one at a time without leaving the model. Define the *depth* of an instance to be $K = \max_i c_i(M)$.

**Why it might work.** The base case is already proved. A depth-one instance has every $c_i$ taking values in $\{0,1\}$, so every bundle costs at most 1 to everybody — which is exactly the hypothesis of Theorem 22. So *Conjecture 2 holds unconditionally at depth* 1, *for every $n$ and every $m$*. No other induction in this project has a proved base case, and none of the failed approaches touches the representation of the cost functions at all: they all manipulate allocations while treating $c_i$ as a black-box oracle.

**Evidence for the inductive step.** Over all 1,330 depth-$\geq 2$ instances at $n = m = 3$, the full instance and its truncation $(\min(c_i, K-1))_i$ *always* share at least one common good allocation — 100%, with no instance where one is solvable and the other is not (`update_10/layers.py`). That is precisely the compatibility an induction on depth needs.

**The kill condition, hunted and not found.** A depth-$\geq 2$ instance whose good allocations are disjoint from its truncation's would end the approach. Across 39,692 depth-$\geq 2$ instances — exhaustive over all dichotomous triples at $n = m = 3$, exhaustive over structured families at $m \leq 4$, and endpoint-constant and deliberately high-depth sweeps to $n = 5$ — there is **no such instance**, in either peeling direction (`update_10/layer_hunt.py`). This is the generator that refuted Proposition 66 on its first run.

The same sweep says which direction to peel. Measuring the fraction of the truncation's good allocations that survive into the full instance, top-peeling $c \mapsto \min(c, K-1)$ carries far more than bottom-peeling $c \mapsto \max(c-1, 0)$ — worst case 0.21 against 0.05 — and on high-depth instances top-peeling carries *all* of them, at $n \in \{3,4,5\}$.

**The strengthened hypothesis, and why it is the right one.** Sharing a witness is necessary but not sufficient: an induction must *name* which good allocation to carry forward. The fix is induction loading — prove something stronger, so that the inductive hypothesis is strong enough to reproduce itself.

**Chain conjecture.** For every dichotomous instance of depth $K$ there is a *single* allocation that is good for $(\min(c_i, k))_i$ *simultaneously* for every $k = 1, \ldots, K$.

Its top level $k = K$ is Conjecture 2; its bottom level $k = 1$ is Theorem 22, already proved. It has **zero failures over** 30,405 **depth-**$\geq 2$ **instances** across the same exhaustive and adversarial sweeps. Being stronger it is a far better induction hypothesis than Conjecture 2 itself, since carrying a whole chain of certificates upward is exactly what the step needs and what the plain statement cannot supply.

**Progress on the step: the chain collapses to two levels.** Write $a_{ij} := c_i(A_j)$ for the cost matrix of an allocation. Every level-$k$ arc depends on the matrix alone,

$$w^{(k)}(i, j) \;=\; \min(a_{ii}, k) - \min(a_{ij}, k),$$

and is therefore *sign-constant and saturating*: it has the sign of $w(i, j)$ throughout, its modulus is non-decreasing in $k$, and it reaches $\left|w(i, j)\right|$ once $k \geq \max(a_{ii}, a_{ij})$. One consequence is immediate and proved.

**Lemma 91** (Level one is exactly envy-freeability)**.** *An allocation is good for* $(\min(c_i, 1))_i$ *if and only if it is envy-freeable for* $(\min(c_i, 1))_i$*. No subsidy condition is needed.*

**Proof** $\min(c_i, 1)$ takes values in $\{0, 1\}$, so every bundle costs at most 1 to every agent and Theorem 22 applies to any envy-freeable allocation. The converse holds because goodness includes envy-freeability. □

Two further statements, tested over 2,288,509 (instance, allocation) pairs — exhaustively at $n = m = 3$, exhaustively over structured families at $m \leq 4$, and randomised to $n = 5$, $m = 6$ — with **zero violations**, but *not* proved:

**(I) Interval.** The set of levels at which a given allocation is good is always an interval.
**(II) Endpoints.** If an allocation is good at level 1 and at level $K$, it is good at every level in between.

Granting (II), the chain conjecture collapses from $K$ conditions to two, and by Lemma 91 one of the two is not a subsidy condition at all:

$$\textbf{Chain conjecture} \iff \exists\, \mathcal{A} \text{ envy-freeable for } (\min(c_i, 1))_i \text{ and good for } (c_i)_i.$$

So the whole approach reduces to finding one allocation that is simultaneously envy-freeable in the *Boolean shadow* of the instance — where every cost is 0 or 1 — and good in the instance itself. Note this is strictly stronger than Conjecture 2: at $n = m = 3$ every top-good allocation happens to be good at all levels, but at larger sizes 3,858 top-good allocations fail level 1, so the Boolean-shadow condition has real content.

**Status and next step.** Proved: the decomposition, the truncation closure, the base case (Theorem 22), and Lemma 91. Open: (I), (II), and the chain conjecture itself. The sign-constant saturating form of $w^{(k)}$ is the handle for (I) — what blocks a one-line proof is that a path mixes positive arcs, which grow with $k$, with negative arcs, which shrink, so path weight is not monotone even though every arc is. A proof of (I) probably needs to track the maximising path rather than a fixed one.

## 14 Conclusion and Future Work

We have not settled Conjecture 2 in general, but Section 11 now carries a construct-and-repair algorithm together with a proof of most of what one would want from it. Four properties are theorems for every $n$: the algorithm's outputs are *certified*, so it never lies (Theorem 81); it always terminates, in polynomial time $O(n^3m^3\tau + n^4m^3)$ (Theorem 82); its search space is genuinely the unordered balanced partitions (Lemma 80); and for *two agents* it is *complete* (Theorem 83) — every two-agent dichotomous instance admits a balanced split of $q$-spread at most 1, found in $O(m\tau)$ time with no restarts. That last result even strengthens the known $n = 2$ chore case, adding a balancedness guarantee the complementation route of Remark 9 does not provide.

For $n \geq 3$ what is missing is completeness alone, and it is backed by **zero observed failures across** 389,215 **test instances**, including every hard instance in this project. So the headline is a proved algorithm whose only unproved property is that it always succeeds — a single, sharply stated combinatorial claim (Conjecture 88), with a proved base case and a named proof technique to generalise.

Getting there required ruling out three strategies a reader would reasonably try first: item-by-item insertion, the strategy that works in [BKNS22] (Theorem 30, and again in the pure goods coordinates of Section 11.3); restriction to utilitarian-optimal allocations, together with *every* refinement of that set (Proposition 33, Corollary 35); and encoding the coverage constraint into the numbers, at either the valuation or the algorithm level (Section 9). Alongside those sit a small set of positive results — binary additive costs (Theorem 18, now subsumed by [LMS26]), identical costs (Theorem 20), two agents — and structural tools that say precisely how the chore problem differs from its goods mirror.

**The lesson that ties the whole investigation together.** Every insertion-based strategy failed for the same reason, in three different coordinate systems: chores directly (Section 6), the replica/peel frame (Section 8), and even the pure goods frame (Section 11.3). Building an allocation one item at a time fixes decisions before their consequences are visible, and by the time a bad grouping shows itself, insertion has no way to undo it — only permutation between agents, never a genuine re-split of a bundle's contents, is available. The algorithm that finally worked abandoned that template entirely: *construct* a whole candidate partition directly, then *repair* it by moves that can freely undo an earlier bad grouping. That is the one idea from this project worth carrying into any future attempt on a similar problem.

The three negative results are more useful together than separately, because they fail in the same way. Insertion fixes an owner before the consequences are visible; a selection rule fixes a criterion before knowing which allocation it will pick out; a reweighting fixes a penalty before knowing who will hold the duplicate. Each is *a rule fixed ex ante against a quantity determined ex post* (Section 9.6). What survives is procedures that decide late — which is exactly the design of the peel frame, where ownership is settled only by the last move touching a chore.

Both negative results are failures of *timing*, and the constraint they jointly impose is that *a correct algorithm must be free to move already-allocated chores between bundles, and free to give up total cost while doing so*. The replica transform of Section 8 is the response: Theorem 38 turns the instance into a goods instance with an identical envy graph, Corollary 40 reduces the

conjecture to the single constraint that no agent be relieved of a chore twice, and reading that dynamically defers every ownership decision to the last move touching a chore, so there is nothing to re-open. This is currently the most promising route. It is not known to work — Proposition 47 shows the state space has genuine dead ends, and Remark 50 notes that the reduction is sufficient but not known to be complete — and what is missing is a peel rule that provably never enters a deadlock.

Every construction here that succeeds does so by exhibiting a per-agent potential $\phi$ with $w_{\mathcal{A}}(i,j) \le \phi_i - \phi_j$, which telescopes along paths. Finding such a potential for general negative dichotomous costs, or proving none exists, remains the problem; the balance signal of Remark 48 is the current best guess at its shape.

**Directions.**

1. **Prove Conjecture 88 for $n \ge 3$** (Section 11.8). This is now the top-ranked direction by a wide margin, and it has a proved base case: completeness holds at $n = 2$ for every $m$ (Theorem 83), constructively and in $O(m\tau)$ time, by the $h$-window technique — a discrete intermediate-value argument on the aggregate $u = \tilde{v}_1 + \tilde{v}_2$ along swap paths, closed off by extremal endpoint arguments. Soundness, certification, termination and polynomial complexity are theorems for every $n$ (Theorems 81 and 82). What remains is the $n \ge 3$ analogue of the window argument: a one-dimensional aggregate whose per-move variation is smaller than the forbidden gap it must cross, controlling $\binom{n}{2}$ pairwise comparisons at once. The 14 known local optima at $n = 3$ bound what any such proof must accommodate, and Remark 87 records the move-set refinement (pairwise exchanges) the proof already forced on the implementation.

2. **The type-ordered peel search** (Section 8.7, Section 12.2). Conjecture 51 has the best evidence of anything currently open and a much smaller state space than Conjecture 49. Run the fixed-point computation over the full $n = m = 3$ dichotomous family; a bad root anywhere kills the approach of Section 8, a clean sweep is strong evidence for it. Cheapest decisive experiment available. Note what a recipient rule must now satisfy: by Theorem 61 the recipient in the hard case has zero marginal, and by Proposition 52 $\arg\max_i \ell(i)$ is insufficient even under the type-ordering, so the rule has to read the residual workload.

3. **Choose the paid set first** (Proposition 14, Proposition 72). The target is exactly: an envy-freeable allocation together with a bipartition of the agents such that all strict envy runs from one side to the other. This is a condition on a bipartition rather than on a schedule, so it inherits none of the dead ends of Section 8 and none of the ex-ante failures of Section 9. It is the one direction that four separate lines of attack have now pointed at and nobody has tried.

4. **Agent induction with a path-based invariant** (Section 10). The arc-based invariant is refuted, so any revival must carry path weights, which are not assignment-invariant. The equality graph of [LMS26, §4.3.1] is the natural candidate for what to carry. Harder than it looked a session ago, but not closed.

5. **Characterise the dead ends** (Remark 48). Test across that family whether every dead end is a state already committed to an unbalanced terminal. If so, the balance rule is the missing ingredient and can be stated precisely.

6. **Settle the converse** (Remark 50). Is a good terminal state, when one exists, always reachable by a legal schedule? A positive answer upgrades the peel frame from a sufficient reformulation to a complete one, and makes the search of item 1 a direct test of Conjecture 2.

7. **Adversarial instance generation** (Section 12.1). The randomised evidence is drawn from a distribution that essentially never produces the structured extremal functions from which both negative results are built. This is the cheapest way to find out whether Conjecture 2 is true at all.

8. **Exchange paths rather than single transfers** (Section 7.2). Single-chore local search has bad local minima. The move set should match the exchange structure of dichotomous costs, as Yankee Swap and matroid union do in the matroid-rank literature [BEF21]; and by Remark 34 the search must range over all allocations, not the utilitarian-optimal ones.

9. **The matroid-rank subclass** (Section 8.8). Write out the supermodular-to-matroid reduction properly, in particular the parallel-extension step, then attempt basis exchange in the parallel-copy matroid. This is where the surrounding literature's machinery is strongest.

10. **Build the two tiers first** (Proposition 14). Choose the partition into paid and unpaid agents and only then allocate. This is the one natural approach that Theorem 30 does not obviously kill, since it is not incremental in the chores. In the proof of Theorem 18 the paid set is exactly the agents holding a $\lceil q/n \rceil$-share of the universally disliked chores; identifying what plays that role in general is the first question.

11. **Climb the milestone ladder.** Binary additive is done (Theorem 18); binary submodular is the next rung, by item 6 above; general dichotomous is the summit. Note that Theorem 15 and Remark 48 agree on what the middle rung needs — cardinality balancing — which is what matroid union would supply.

12. **The mixed model.** Allow each item to be a good or a chore for each agent, with all marginals in $\{-1, 0, 1\}$: the dichotomous restriction of the doubly monotone setting of [KMS+24]. This is the natural common generalisation of [BKNS22] and of the present paper.

**Where this would sit.** Conjecture 2 is the fourth corner of the square in Section 1: additive goods [BDN+20], dichotomous goods [BKNS22] and additive chores [LMS26] all give one dollar per agent and $n - 1$ in total, and dichotomous chores is what remains. For valuations that are dichotomous but not additive the best published bound is still that of [KMS+24], $n - 1$ per agent and $n(n-1)/2$ in total from any EF1 allocation, so the conjecture would improve the total by a factor of $n$ — the same factor [BKNS22] gains over [BDN+20] on the goods side.

Two cautions follow from the literature check. First, Theorem 18 is already subsumed by [LMS26] (Remark 19); what survives of it is the mechanism, not the statement. Second, [LMS26] appeared in July 2026, which is a reminder that the surrounding corners are being filled in quickly and that the search should be repeated before circulation rather than treated as done.

## References


[BCE+16] Felix Brandt, Vincent Conitzer, Ulle Endriss, Jérôme Lang, and Ariel D. Procaccia. *Handbook of Computational Social Choice*. Cambridge University Press, 1st edition, 2016.

[BDN+20] Johannes Brustle, Jack Dippel, Vishnu V. Narayan, Mashbat Suzuki, and Adrian Vetta. One dollar each eliminates envy. In *Proceedings of the 21st ACM Conference on Economics and Computation (EC)*, pages 23–39, 2020.

[BEF21] Moshe Babaioff, Tomer Ezra, and Uriel Feige. Fair and truthful mechanisms for dichotomous valuations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35. AAAI, 2021.

[BKNS22] Siddharth Barman, Anand Krishna, Y. Narahari, and Soumyarup Sadhukhan. Achieving envy-freeness with limited subsidies under dichotomous valuations, 2022.

[BMS05] Anna Bogomolnaia, Hervé Moulin, and Richard Stong. Collective choice under dichotomous preferences. *Journal of Economic Theory*, 122(2):165–184, 2005.

[BSV21] Umang Bhaskar, A. R. Sricharan, and Rohit Vaish. On approximate envy-freeness for indivisible chores and mixed resources. In *Approximation, Randomization, and Combinatorial Optimization (APPROX/RANDOM)*, LIPIcs, 2021.

[BSY23] Xiaolin Bu, Jiaxin Song, and Ziqi Yu. EFX allocations exist for binary valuations. In *Proceedings of the International Conference on Frontiers of Algorithmic Wisdom (FAW)*, 2023.

[Bud11] Eric Budish. The combinatorial assignment problem: Approximate competitive equilibrium from equal incomes. *Journal of Political Economy*, 119(6):1061–1103, 2011.

[CI21] Ioannis Caragiannis and Stavros Ioannidis. Computing envy-freeable allocations with limited subsidies. In *Workshop on Internet and Network Economics (WINE)*, 2021.

[DCW+24] Sijia Dai, Yankai Chen, Xiaowei Wu, Yicheng Xu, and Yong Zhang. Weighted envy-freeness in house allocation, 2024.

[End17] Ulle Endriss. *Trends in Computational Social Choice*. Lulu.com, 2017.

[Fol67] Duncan Karl Foley. Resource allocation and the public sector. *Yale Economic Essays*, 7(1):45–98, 1967.

[GIK+21] Hiromichi Goko, Ayumi Igarashi, Yasushi Kawase, Kazuhisa Makino, Hanna Sumita, Akihisa Tamura, Yu Yokoi, and Makoto Yokoo. Fair and truthful mechanism with limited subsidy, 2021.

[GSW25] Jugal Garg, Eklavya Sharma, and Xiaowei Wu. Proportional and Pareto-optimal allocation of chores with subsidy, 2025.

[HS19] Daniel Halpern and Nisarg Shah. Fair division with subsidy. In *Proceedings of the 12th International Symposium on Algorithmic Game Theory (SAGT)*, volume 11801 of *Lecture Notes in Computer Science*, pages 374–389. Springer, 2019.

[KMS+24] Yasushi Kawase, Kazuhisa Makino, Hanna Sumita, Akihisa Tamura, and Makoto Yokoo. Towards optimal subsidy bounds for envy-freeable allocations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[LMMS04] Richard J. Lipton, Evangelos Markakis, Elchanan Mossel, and Amin Saberi. On approximately fair allocations of indivisible goods. In *Proceedings of the 5th ACM Conference on Electronic Commerce (EC)*, pages 125–131, 2004.

[LMS26] Xinhang Lu, Simon Mackenzie, and Mashbat Suzuki. Optimal subsidy bounds for goods and chores: One dollar each suffices, 2026.

[Mas87] Eric S. Maskin. On the fair allocation of indivisible goods. In G. Feiwel, editor, *Arrow and the Foundations of the Theory of Economic Policy*, pages 341–349. MacMillan, 1987.

[NSV21] Vishnu V. Narayan, Mashbat Suzuki, and Adrian Vetta. Two birds with one stone: Fairness and welfare via transfers. In *International Symposium on Algorithmic Game Theory (SAGT)*, pages 376–390. Springer, 2021.

[TWYZ23] Biaoshuai Tao, Xiaowei Wu, Ziqi Yu, and Shengwei Zhou. On the existence of EFX (and pareto-optimal) allocations for binary chores, 2023. Journal version in Theoretical Computer Science, 2025.

[Var74] Hal Varian. Equity, envy, and efficiency. *Journal of Economic Theory*, 9(1):63–91, 1974.